

Machine Learning with Confidential Computing: A Systematization of Knowledge

FAN MO, Imperial College London, London, United Kingdom of Great Britain and Northern Ireland

ZAHRA TARKHANI, Microsoft Research New England, Cambridge, United Kingdom of Great Britain and Northern Ireland

HAMED HADDADI, Imperial College London, London, United Kingdom of Great Britain and Northern Ireland

Privacy and security challenges in Machine Learning (ML) have become increasingly severe, along with ML's pervasive development and the recent demonstration of large attack surfaces. As a mature system-oriented approach, Confidential Computing has been utilized in both academia and industry to mitigate privacy and security issues in various ML scenarios. In this article, the conjunction between ML and Confidential Computing is investigated. We systematize the prior work on Confidential Computing-assisted ML techniques that provide (i) *confidentiality guarantees* and (ii) *integrity assurances* and discuss their advanced features and drawbacks. Key challenges are further identified, and we provide dedicated analyses of the *limitations* in existing *Trusted Execution Environment* (TEE) systems for ML use cases. Finally, prospective works are discussed, including grounded privacy definitions for closed-loop protection, partitioned executions of efficient ML, dedicated TEE-assisted designs for ML, TEE-aware ML, and ML full pipeline guarantees. By providing these potential solutions in our systematization of knowledge, we aim to build the bridge to help achieve a much stronger TEE-enabled ML for privacy guarantees without introducing computation and system costs.

CCS Concepts: • **Computing methodologies** → **Machine learning**; • **Security and privacy** → **Trusted computing**; *Software and application security*; *Privacy-preserving protocols*

Additional Key Words and Phrases: Privacy-preserving machine learning, confidential computing, trusted execution environment

ACM Reference Format:

Fan Mo, Zahra Tarkhani, and Hamed Haddadi. 2024. Machine Learning with Confidential Computing: A Systematization of Knowledge. *ACM Comput. Surv.* 56, 11, Article 281 (June 2024), 40 pages. <https://doi.org/10.1145/3670007>

1 Introduction

Machine learning (ML) has been established as the most promising approach to learning patterns from data, and its applications exist across data processing of online surfing [1, 2], financial data [3], health care [4, 5], autonomous cars [6, 7], and almost every data-driven field around us. The wide applications have also led ML models, specifically **Deep Neural Networks (DNNs)**, to be run on a

Authors' Contact Information: Fan Mo, Imperial College London, London, United Kingdom of Great Britain and Northern Ireland; e-mail: mofan1992@gmail.com; Zahra Tarkhani, Microsoft Research New England, Cambridge, United Kingdom of Great Britain and Northern Ireland; e-mail: zt233@cantab.ac.uk; Hamed Haddadi, Imperial College London, London, United Kingdom of Great Britain and Northern Ireland; e-mail: h.haddadi@imperial.ac.uk.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](https://permissions.acm.org).

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 0360-0300/2024/06-ART281

<https://doi.org/10.1145/3670007>

diverse set of devices—from low-end IoT/mobile devices to high-performance cloud/data centers to provide both ML model training and inference services. However, the wide applications also open a large attack surface on ML's security and privacy due to a large number of mistrusting entities and complex software/hardware infrastructure involved. Recent research has massively explored the way to attack ML. Several examples are, e.g., model stealing [8–11], model inversion [12, 13], model/data poisoning [14–16], data reconstruction [17, 18], membership/attribute inference [19–21], and so on. Vulnerabilities shown by these attacks have increasingly perturbed ML developers and users due to the negative consequences they can cause: the destruction of their ML training process, the leakage of private information, or the infringement of intellectual property.

Specifically, ML faces security and privacy issues mainly due to, first, the *complexity of pipelines* that involve many system/software stacks to provide modern features such as parallelization and acceleration. A full ML pipeline covers the collecting of raw data, the complete training and inference phases, the later use of the trained ML model for prediction, and the potential re-train and re-use of the ML model. Because the involved participants, i.e., data owners, the host of ML computation, the model owners, and the result receivers are most likely to be different entities, the pipeline can be segmented across mutually mistrusting individuals and, consequently, is left with a broad attack surface. Second, the *weak robustness and low interpretability* of ML algorithms aggravate the security and privacy problems. The modern ML model (i.e., DNN) is a large-scale nonlinear system that has been progressively adjusted using **stochastic gradient descent (SGD)** during training. Until now, there is still a lack of an accurate representation/description of the dynamics of this training progress, and it has been increasingly hard in terms of model explainability along with the evolving of complicated training techniques and the size of large models (e.g., large language models GPT-4 or Llama). Consequently, ML models often have relatively weak robustness, which has been shown in adversarial examples [22] or poisoning attacks [15], i.e., a small malicious change to the training process or input data could cause large negative impacts that are difficult, or even impossible, to detect.

Among many candidates to achieve trustworthy ML, a system-oriented solution that provides a trusted environment within the system, called Confidential Computing, has emerged as one promising approach because of its high performance. Confidential Computing,¹ defined in the Linux Foundation project Confidential Computing Consortium's whitepapers [27, 28], is the protection of data in use, in addition to data protection in storage and transmission by utilizing hardware-based **Trusted Execution Environments (TEEs)**. TEE is one of the emerging techniques in enabling isolated and verifiable code execution inside protected memory (sometimes called enclaves or secure world, depending on realization and manufacturers), separated from the host privileged system stacks such as operating system or hypervisor. Both hardware and software changes are typically required to achieve proper isolation on modern processors (i.e., CPUs) at an adequate trustworthiness level. Compared to the conventional external secure processor, like the Trusted Platform Module that uses separated hardware from the rest of the board/SoC, TEEs still remain and run on existing processors. This delivers the flexibility in switching contents between normal execution mode and trusted execution mode and further provides much more computational resources in the system. Because of their flexibility and reasonable performance in supplying trustworthiness, TEEs have become nearly an essential component of modern processors, and various types of TEEs are implemented by, e.g., ARM, Intel, AMD, and NVIDIA, to realize Confidential Computing.

¹There are other cryptography-based techniques such as Multi-party Computation and (Leveled or Full) Homomorphic Encryption used for guaranteeing ML confidentiality and integrity; they are categorized as Privacy-preserving Computation to distinguish with TEE-based Confidential Computing [23–26].

The rapid growth of Confidential Computing pushes a line of research work that leverages this technique for trustworthy ML computations [29–35], and a great number of commercial products are available nowadays, e.g., Confidential Computing service from Microsoft Azure [36], Amazon Web Services [37], Google Cloud [38], and so on. Although many attempts have been made to combine Confidential Computing and ML and several limitations (e.g., still limited TEE resources for ML use) are summarized [39], there is still no systematization of Confidential Computing-assisted ML, challenges, and especially potential solutions in a long-term vision. There are still unclarities of what the critical scientific and engineering obstacles are and how to overcome them to achieve trustworthy ML. Furthermore, both Confidential Computing and ML have complicated mechanisms, respectively; a lack of such systematized analysis makes a sophisticated combination of these two impossible, which even further harms both the integrity of Confidential Computing and the performance of ML.

To alleviate the security and privacy problems in ML, in this article, we present a systematic analysis of Confidential Computing-assisted solutions.

Organization. The next sections of the article are organized as: Section 2 focuses on ML, and it describes the pipeline and paradigms of ML and existing attack surfaces; Section 3 focuses on Confidential Computing, and it mainly outlines the threat model and key components of Confidential Computing; Section 4 summarizes challenges and existing solutions when using Confidential Computing to provide trustworthy ML, specifically in terms of confidentiality; Section 5 further presents the challenges and solutions of guaranteeing ML *integrity* with Confidential Computing; Section 6 outlines the existing TEE limits in Confidential Computing that hinder its usage in ML; Section 7 concludes the current research stage and further work.

2 Machine Learning

ML automates pattern learning from large datasets in the sense that analysts do not need to manually explore data's latent features and their correlations [40, 41]. In this section, we look into ML pipelines, common paradigms, and attack surfaces that exist among them.

2.1 Pipeline

The ML pipeline codifies and automates the workflow to produce and apply ML algorithms or models. It consists of multiple sequential steps forming the lifecycle of using ML. We present the general ML pipeline in Figure 1. The form can change slightly, depending on different ML use cases, but the core parts are interchangeable overall, as identified by works from Microsoft [42], IBM [43], and ETSI [44]. Key stages of the ML pipeline are given below.

Data preparation. Data are the basis for ML evidently, because ML requires tons of inputs to attempt “trial and error” and learn the patterns. In supervised learning [41], all data are labeled in some ways, which conventionally need extensive efforts of human annotation. In semi-/unsupervised learning [45–47], parts/all of the data are unlabeled due to the difficulty of labeling. In some cases, unlabelled parts of data can be “self-learned” after the model already gains a certain level of prediction ability on these data [48]. Also, no label is needed when the model aims at learning encoding features and then decoding features back to original data (e.g., Autoencoder [49, 50]). In reinforcement learning [51], data are in the form of sequences of action, observations, and rewards, which are sequentially produced/updated during the learning process of a policy function. Regardless of the form, ML is always *data-hungry*. Thus, many efforts have to be made to obtain a fair amount of data to cover possible features that the data have in their distribution space. One common technique to increase the amount of data is data augmentation, which adds slightly modified copies or synthetic copies of existing data [52, 53]. In addition, another critical issue is that the collected dataset can be very noisy or contain bad records/features from which

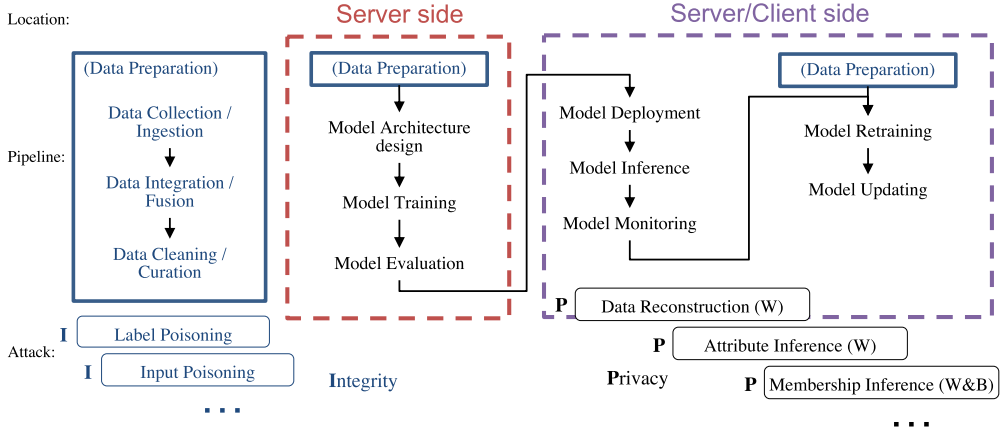


Fig. 1. The schematic diagram of the machine learning pipeline and resided attack surfaces across the pipeline. “I” refers to Integrity-related attacks. “P” refers to Privacy-related attacks. “W” refers to that one attack is generally in white-box form, while “B” refers to black-box form.

the ML model should not learn. Then, it is almost always necessary to apply filter-like algorithms (e.g., data clean or data selection schemes) to refine the data or select training data by measuring the importance of data samples [54–56]. After that, collected data are typically partitioned as the training set and the test set (sometimes additionally, the validation set) for training and evaluating the ML model.

Model training. Most model training aims to acquire a function $f_{\theta}(x)$ for $\theta \in \Theta$, capable of mapping an input x to a predicted decision $\hat{y} = f_{\theta}(x)$ near to the true decision y , where θ and Θ denote model parameters and its corresponding parameter space. The form of $f_{\theta}(\cdot)$ other than θ and implicitly included in function $f(\cdot)$ is called model hyperparameters; specifically regarding DNNs, hyperparameters are determined by chosen DNN architectures, e.g., the type of layers, the number of neurons, and so on. Note that the underlying hyperparameters also affect the training performance (sometimes a lot), which can be tuned manually or automatically using neural architecture search [57]. Nevertheless, model training, conventionally, regards the searching for the best θ set that is achieved by minimizing the loss $\ell(y, f_{\theta}(x))$ using SGD methods [58, 59] on training sets, i.e., stepped descent by $\theta \leftarrow \theta - \eta \nabla_{\theta} f_{\theta}(x)$, where η is the step size multiplier known as the learning rate. To update θ of the complete DNN model, the loss is backward passed through the DNN starting from the last layer to the first, which is called *backward propagation*. After reaching a certain level of performance on the validation set (or the other subset apart from the training set in cross-validation [60, 61]), the model training stops. Finally, $f_{\theta}(\cdot)$ ’s final model utility/accuracy is reported on the test set.

Model deployment/Inference. With the above trained DNN model, one can feed unseen input data x_{new} into the DNN model to get the prediction $\hat{y} = f_{\theta}(x_{\text{new}})$. This is called the inference stage. It is worth noting that the inference could happen at both the *model provider* side or the *data owner* side, depending on the trust and collaboration form between the providers and the users. For instance, in typical cases of ML inference, the model needs to be deployed (i) on a server-side device to support centralized service, e.g., ML inference as a service or (ii) on the users’ devices for distributed local inference.

Retraining/Updates. To adapt to data with unseen features over time, ML models require retraining/updating their parameters. Such updates can be achieved by fine-tuning the model with newly

collected data in some forms. Especially, the fine-tuning could focus on the last layers of the model, which is one way to achieve transfer learning [62, 63], as the first layers already generalize well on one type of data (e.g., images in general) [62, 64, 65] to improve the training efficiency. Two examples of such a technique are (i) back-propagation on the last layers only [63] and (ii) low-shot learning with weights imprinting on the last layer [66]. The reason for updating the last layers preferentially is related to one of DNN's learning behaviors; its first layers tend to learn more general features that all input data should have, while the last layers learn more specific features that do not cover some new incoming data [62, 64, 65]. However, whether we can take such a "shortcut" or not depends on how new the data's features are with respect to the pre-trained DNN model. Similarly to model inference, this stage can happen on not only the server side but also the client side, depending on the cases. Retraining on the server usually requires newly collected data that may be from clients. In the other case, after having an updated model on the client side, it can further share it with other users or the central server for knowledge sharing, sometimes called distributed learning or federated learning [67–69].

2.2 Paradigm

The stages of the ML pipeline are typically distributed among different participating entities. Such distribution of ML process is partially due to the increasing privacy/security concern raised by sharing data, as well as network/computation-resource constraints, and therefore, aims at a better tradeoff among model utility, privacy preservation, and system performance and communication cost. Based on the location of the main ML computation (e.g., training or inference), we categorize them based on the well-accepted concept: *centralized machine learning* and *distributed/federated machine learning*.

Centralized machine learning. In centralized ML, *all* of the training or inference computations are conducted at a central place such as a server [70, 71]. For both training and inference, the data have to be collected and stored at/near the center, and then the ML algorithm performs training or inference functions on the data. In such a case, potential retraining will also happen on the server with newly collected data. Centralized ML usually achieves the best training performance compared to all forms of federated ML, because it has all data on hand with the probability of attempting various training strategies that are impossible when data are distributed. Nevertheless, centralized ML raises privacy concerns about others' data, considering that all these data are required to be gathered together. Therefore, centralized training/inference is usually chosen for the cases where collected data are public (or non-private), or the data contributors are collaborators or have *some* form of trust/contracts with the central server.

Distributed machine learning. In the other cases, when data owners do not have the willingness to share data, federated ML is one potential approach where the *ML model*, instead of the private data, is shared among participating entities [67, 72]. Such an ML paradigm enables the client to perform computations on the model and collaborate with other clients with some specifically designed protocols [73, 74]. Specifically, in terms of *ML inference* in a distributed way, it is quite straightforward that the model owner *distributes* its ML model to end-users, so users can perform inference locally on their devices. Potentially, they can further conduct fine-tuning on the model for personalization but still for their own usage. Regarding the *ML training*, the terminology "**federated learning (FL)**" [67–69] has been intensively used referring to the paradigm that users send the locally updated model parameters to the server without the need of revealing their data. User's updated models are gathered and then aggregated as one global model on the server, using chosen aggregation techniques, e.g., FedProx [75], FedAdam [76], FedMA [77], and so on. Unfortunately, despite that such FL designs never share local data, recent works demonstrate the probability of retrieving some form of sensitive information (e.g., attributes of the local data) from

the model parameters themselves [17, 20, 78]. However, recent research reveals that even in federated learning, the shared model parameters still make the system vulnerable due to information leakage and potentially malicious changes in model parameters. The taxonomy of ML paradigms is further clarified when elaborately describing the underlying protection scenarios in Section 4.1 and Figure 4.

2.3 Attack Surface

The attack surface can cover the complete pipeline caused by the potential multiple vulnerabilities mentioned above. Here, we categorize existing attack vectors based on the core underpinning of information security into (i) *confidentiality*, i.e., privacy of data and intellectual property of models, (ii) *integrity* of the ML process. Note that while the full CIA triad includes Confidentiality, Integrity, and Availability, the attacks that specifically aim at ML system's Availability, e.g., **Denial-of-Service (DoS)** attacks, have not drawn much attention from both industry and academia currently, probably due to the high similarity of availability problems between conventional systems and ML systems. Further note that, despite their differences, we use *confidentiality* and *privacy* interchangeably, sometimes considering that the confidentiality of the data or model usually represents the privacy of their owners. Thus, the attacks on every vulnerability in the ML pipeline are viewed in light of confidentiality and/or integrity.

Confidentiality-related attacks. An adversary can be honest-but-curious, i.e., honestly performing ML training/inference without changing the computation results, but actively explore (unauthorized) sensitive information of data and the model. Such attack vectors exist across stages of the ML pipeline including model training, model deployment/inference, or model updates, because the data and models are typically owned by multiple mistrusting entities. In terms of centralized ML, the computation host can directly access incoming data [79, 80] without any protection schemes, thus violating the privacy of data. In federated ML, the host who orchestrates all clients' local training can access their updated models and further infer private information about their local data using these models [20, 78, 81], although it cannot directly access clients' private data. Common attacks on private data can be categorized into (i) the **Data Reconstruction Attack (DRA)** [17, 78, 82], aiming at reconstructing original input data based on the observed model or its gradients, (ii) the **Attribute Inference Attack (AIA)** [20, 83], aiming at inferring the value of users' private properties in the training data, and (iii) the **Membership Inference Attack (MIA)** [19, 81], aiming at learning whether specific data instances are present in the training dataset. These three categories thoroughly cover the spectrum of privacy-related attacks. DRAs lead to complete privacy leakage about target data, while MIAs are the mildest with leakage of data's membership information only. AIAs broadly cover middle levels of privacy leakages.

On the other side, the confidentiality of models is also targeted by attacks. One model owner usually considers a model as its intellectual property, as the model has been trained with great effect (e.g., computation power, data collection, and data cleaning work), such as the pre-trained model in **machine learning as a service (MLaaS)** hosted by **Amazon Web Services (AWS)**. One well-explored type of attack, called the **Model Stealing Attack (MSA)**, aims to counterfeit the functionality of a model by exploiting black-box access in ML inference [8, 9]. One example is to query a large number of prediction results and then use these input-output pairs to solve "equations" or "retrain" a model somehow [9, 84]. This problem becomes severe when the model size is smaller and sometimes related to highly valuable training data that are hard to obtain.

Integrity-related attacks. In addition, attackers can actively exploit training/inference results, thus, violating the integrity of the ML process. The fairly most well-known adversarial attack, called **Adversarial Examples (AE)** [22, 85], adds calibrated noises to one image/audio, consequently leading to wrong prediction results during inference. The input with noisy perturbation is

invisible to human beings, making such attacks really hard to detect. In a more influential case, attackers take full control of wearable **BCI (Brain computing interface)** devices (e.g., wheelchair, robotic arms, or drone) by compromising input signals to on-device ML models [86], which sometimes can be life-threatening. Besides, previous work has also shown the possibility of model accuracy degradation by only compromising thread scheduling on multi-threaded ML pipeline [87]. We note that although these attacks manipulate input data, they break the integrity of ML inference.

Regarding ML training, specifically in federated learning, **Model Poisoning Attacks (MPA)** can lead to Byzantine fault [14]. Such faults easily occur, because the server does not control clients' local training and cannot verify whether their behaviors have followed the promised training processes. For instance, if one error in values of locally updated parameters has been aggregated into the global model, then the complete global model can become unusable or even fail to converge. In a more sophisticated way, one can manipulate the training set, e.g., by adding data with calibrated noisy labels [88] to drive the training process into undesired ways, namely, Backdoor Attacks or **Data Poisoning Attacks (DPA)** in some works. This type of attack fools the classifier into having wrong boundaries for some specifically chosen data points. The attackers/other users will be able to trigger such backdoors in later use after model deployment [89–91].

White-box or black-box. Here, we further describe a common way of categorizing ML attacks: white-box and black-box. Naturally, adversaries have different levels of prior information before performing attacks. Here, ML attack surfaces can be categorized based on whether one attack requires access to the internal architecture of an ML model [92, 93]. All above-mentioned attacks can be grouped into white-box or black-box based on the level of required knowledge of the target ML model. With regards to black-box attacks, model stealing usually starts from the outside without any prior information and aims to learn the model itself. Also, membership inference attacks on data privacy are typically black-box, because it is already much more efficient than white-box attacks, as shown in previous research [93, 94]. White-box access does not significantly increase the attack “advantage” in disclosing membership privacy. On the other side, the white-box attack includes almost all data reconstruction attacks, some adversarial example attacks, and attribute inference attacks. These attacks usually require detailed model parameters/gradients to be performed or have reasonable performance. We refer to Papernot et al.'s survey [92] and NIST's publication [95] for more details about the taxonomy and terminology of ML's security and privacy vulnerabilities.

3 Confidential Computing and Trusted Execution Environments

Confidential Computing [26] is a quickly emerging technology that leverages **Trusted Execution Environments (TEE)** to provide a level of assurance of privacy and integrity when executing codes on data. Most Cloud service vendors, e.g., Google Cloud and Microsoft Azure, have now started providing Confidential Computing services, which usually use the TEE solution supplied by processor manufacturers such as ARM, Intel, AMD, and NVIDIA [96]. In this section, we first describe threat models considered in Confidential Computing and the key components of it. Then, we discuss the software stacks useful for ML application developers.

3.1 Threat Models

By considering a stronger adversary who has full access to the host privileged system stacks [27], Confidential Computing significantly reduces the **trusted computing base (TCB)**. By default, TEEs assume that the attacker controls the host OS/hypervisor as well as the host applications executing in userspace. She could, for example, access and corrupt the host OS kernel and process resources (e.g., memory, threads, network, files, and locks). Hence, the adversary can be the

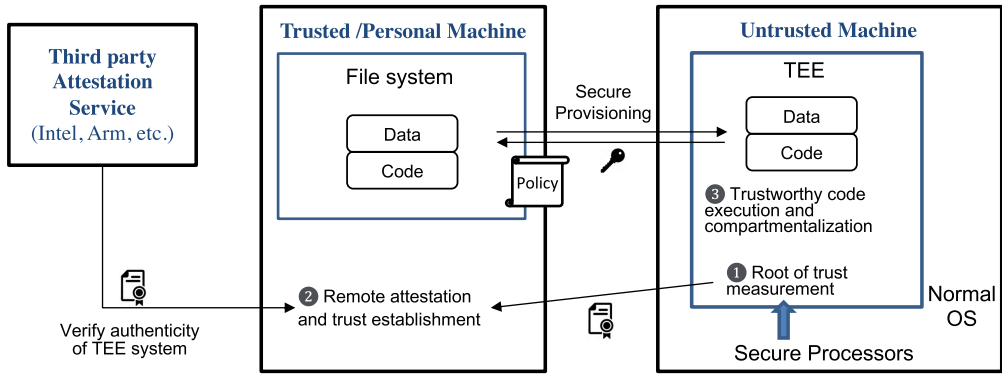


Fig. 2. The schematic diagram of Confidential Computing by utilizing the Trusted Execution Environment (TEE).

service provider, the device owner itself, malicious third-party software installed on the devices, or a malicious or compromised host OS/hypervisor. Under such an environment, they can perform any possible white-/black-box or confidentiality-/integrity-related attacks.

Confidential Computing enables trust between mutually mistrusting components by utilizing (preferable hardware-based, attested) TEEs [97]. Chip manufacturers usually have different realizations for their own chip structure. As an example, ARM designs its TEEs as an isolated environment running in parallel with the host OS (also known as the rich OS), allowing to securely store data and execute arbitrary code on an untrusted device, with reasonably good performance, through protected and measured memory compartments. Thus, one can demand a remote TEE to perform computation on her (sensitive) code and/or data without revealing them to the TEE's host. Few TEEs, e.g., AWS Nitro Cards [37], could also extend the level of hardware-based isolation to peripherals and persistence storage [98, 99] via a range of trusted PCIe interfaces or TEE-IO/device interfaces such as **TDISP (TEE Device Interface Security Protocol)**, **IDE (Integrity and Data Encryption)**, **SPDM (Secure Protocol and Data Model)**, which consequently extend the trust boundary to cover more system components [100].

The recent commoditization of TEEs both in high-end and low-end mobile devices makes it an ideal candidate to achieve confidentiality and integrity in ML. However, if the code inside an enclave has vulnerabilities, then TEEs do not guarantee the protection of the integrity and confidentiality of that computation. Consequently, the attacker may be able to exploit weaknesses in TEEs system [101] and their interfaces/APIs in such conditions [102, 103].

However, TEEs are vulnerable to various side- and covert-channel attacks [104, 105], physical attacks [106], confused deputy attacks [107], DoS attacks, and so on [108, 109]. These attack vectors are hard to defend against in the standard TEE threat models [110] and are considered out-of-scope attacks, although we briefly discuss these attacks and corresponding defenses in the discussion section.

3.2 Key Components of Confidential Computing

We give an overview of the Confidential Computing structure in Figure 2, then present key components of it below.

Root of trust measurement. Confidential Computing provides hardware-assisted functionalities to establish trust between TEEs and the external computing environment. First, by enabling **Root of Trust (RoT)** measurement it ensures the integrity of the TEE system [97, 125]. When the

processor measures the chain of all critical system software (including boot loader, firmware, OS, hypervisor, and TEE system) before launching in-enclave code, any integrity violation of the initial state could be detected. A compromised code in the chain cannot escape from being measured and therefore from being detected. The measurement needs hardware unique keys that will be attested later to ensure the integrity of the TEE system [97, 125].

Remote trust establishment and attestation. Before deploying software or data into TEEs, one needs to remotely attest the TEE system's integrity. Remote attestation [125, 126] allows the user to determine the level of trust/integrity of a remote TEE before transmitting her sensitive data/code. It enables the user to authenticate the hardware, verify the trusted state of the remote TEE, and check whether the intended software is securely running inside the TEE. The authentication starts with signing the RoT measurement report with a private key that is securely stored on the system's hardware. The report then is sent to the attestation service, which verifies the signature using the system's public key, and checks the report against a trusted reference, e.g., a known-good configuration of the system. The attestation service then generates an attestation certificate that contains a signed statement of the system's integrity. Finally, the attestation certificate is sent to the attester. Attestation could be conducted by a third party besides the user and host [127] or directly between the service provider and enclave owner [128]. For example, the attestation server could be launched by the processor manufacturer/vendor, as in Intel's SGX attestation service [129], which can be different from the host service provider (e.g., a cloud provider). Nitro Enclave provided by AWS uses its own remote attestation service (e.g., AWS Key Management Service). In such a case, one AWS Nitro Enclave customer provides assurances to their downstream customer.

Trustworthy code execution and compartmentalization. The core feature provided by Confidential Computing is the hardware-assisted isolation of enclaves/TEEs from the untrusted environment [97]. In general, enclave memory regions are protected and managed through hardware and TEE's system software. Isolation mechanisms for processor architectures can be different (e.g., for ARM, Intel, and AMD). For instance, SGX relies on **Memory Encryption Engine (MEE)** to protect the confidentiality, integrity, and freshness of the CPU-DRAM traffic over enclave memory ranges, while TrustZone relies on separate page tables, hardware privilege layers (e.g., EL3 and Secure EL2/EL1/EL0), and TrustZone address space controller (TZASC).² More importantly, they run enclaves in different privilege and execution modes. For instance, SGX enclaves run in userspace mode, while AMD's SEV and Intel TDX [130] run them on hypervisor mode, and TrustZone secure world runs as a virtual separated core from the host core.

Trusted IO and Device Interfaces. Moreover, depending on the TEE/enclave architecture and threat model, it can support additional peripheral compartmentalization and persistent storage sealing. In Table 1, we summarize common hardware-assisted TEEs with their providers, supported processors, and features. Among them, TrustZone [131] is developed from a very early period, and it is the most widely used TEE on mobile/ubiquitous devices. However, it has several constraints, including the inflexible protected memory, supporting only one single secure world, and restricted third-party application adaption. To overcome these issues, recently ARM proposed **Confidential Compute Architecture (CCA)** [132, 133], which aims to run more flexible enclaves (called realm) in parallel with TrustZone. Therefore, modern TEEs support multi-enclaves and more flexible secure memory sizes with different TCBs. TEE systems that include an OS-like software stack usually have a large TCB size, which provides higher usability to developers and users. For

²There are software/language-based TEEs and compartmentalization techniques without the needs of hardware-based guarantees (e.g., seL4), but it is not included here, as they usually have lower security assurance and are not mainstream in Confidential Computing.

instance, when TEEs provide such an OS, they statically link applications into the in-enclave *kernel*; consequently, unmodified applications could be deployed into enclaves. In another line, CCA will be introduced in Armv9-A platforms, while billions of Armv8-A powered devices are still on the market. To tackle such a problem, IceCap, an internal project in Arm, supports a form of isolate on compatible Armv8-A devices by providing a hypervisor with a negligible, attestable TCB based on the high-assurance seL4 microkernel [134]. VirtCCA further implements a virtualized CCA on platforms equipped with the Secure EL2 extension available from ARMv8.4 and earlier platforms that lack Secure EL2 support with minimal overload [135]. However, as mentioned, there are still many open problems such as behavioral measurement and attestation in addition to the TEE itself.

There has also been progress recently on speeding up computation-heavy TEEs with new interface technologies. For example, SEV-TIO builds upon various industry standards such as PCIe's TDISP, IDE, SPD, and others. It establishes a secure connection between a device and a specific SEV-SNP VM, ensuring both confidentiality and integrity of communication. Such a design allows devices to attest guest VMs, preventing interactions with malicious devices, and facilitates high-performance DMA without bounce buffering, while previously, device communication relied on shared (unencrypted) memory pages, requiring data copying into specific pages before device access [136]. There is also TEE/OPTEE integration of NXP's **Resource Domain Controllers (RD-C/XRDC/TRDC)** that provide support for the isolation of peripherals and memory [137]. All these designs could achieve a more flexible/effective compute pipeline with TEEs.

3.3 Partitioning Frameworks for Developers

With the low-level hardware and system stacks only, it is still hard for application developers to utilize TEEs. Hence, depending on the use cases, various software vendors provide deployment and partitioning frameworks for running a diverse set of workloads within enclaves. For example, more recently, Amazon Nitro enclaves and Enarx support running the same binary within different types of TEEs through in-enclave **WebAssembly (WASM)** support, inspired by Haven [138] that ports Drawbridge [139] (a Windows library OS) inside an SGX enclave. In line with above-mentioned IceCap, Veracruz is designed to further support collaborative privacy-preserving computations on multiple platforms including CCA by providing minimal filesystem, i/o, and in-enclave WebAssembly [134, 140]. Similarly, Graphene-SGX [141] ports the Linux-based Graphene library OS in an enclave. Scone [142] tries to reduce TCB by porting musl libc and a portion of **Linux Kernel Library (LKL)** [143]. TrustShadow [98] also protects unmodified applications from the host OS by trapping application exceptions and system calls inside TrustZone, passing the calls to the OS, and verifying the output after parameter marshaling. Other widely used TEE-enabled deployment frameworks include confidential containers and Kubernetes services, which are especially popular and easy to use for ML workloads, as offered by cloud providers such as Azure confidential Kubernetes Service [144] and Google Confidential GKE Nodes [145].

As another approach, various TEE partitioning frameworks split applications into trusted and untrusted components, e.g., Intel SGX SDK [146], Microsoft Open Enclave [147], Google's Asylo [148], OP-TEE [113], and Keystone [149]. There are also language-specific partitioning frameworks such as Civet [150] for porting Java classes into SGX enclaves, Trusted Language Runtime [151] for running portions of C# applications inside TrustZone, and Glamdring [152], a compiler for partitioning applications into SGX enclaves via code annotation. Table 2 summarizes primary commercial TEE frameworks. However, not all of them are feature-rich enough to support ML use cases. Particularly, mobile vendors (e.g., TEE of Qualcomm [153], Trustonic [154], or Huawei [155]) also only allow for limited TEE operations to avoid security risks inside TrustZone secure world [107]. Besides, such security concerns link to the less-privileged execution of enclaves as well as ease of programming.

Table 1. Summary of Dominant Hardware-assisted Trusted Execution Environments

Mainstream Hardware TEEs	ISA & Year	Supported Processors	Number of Isolation	TCB Size (excluding CPU/SoC package) ^α	Secure Memory Size ^δ	Attestation	Application	Application Unchanged	Protections
Software Guard Extensions (SGX)	Intel (2013)	6th Intel CPU +	multi-enclaves	small.	up to 128 MB	remote Intel service	desktop-level	○	confidentiality integrity
SGX 3rd & 4th Gen	Intel (2022/23)	Xeon Scalable CPUs	multi-enclaves	large.	up to 512 GB 1 TB if using XTS	various re-mote services	desktop-level	●	confidentiality integrity
TrustZone (TZ)	Arm (2005)	ARMv6-A + / Armv8-M +	single secure world	small.	typically up to 16 ~ 64 MB [111]	FTPM-based	mobile-level desktop-level	○	confidentiality integrity
Secure Encrypted Virtualization (SEV)	AMD (2016)	AMD EPYC +	multi-VMs	large.	up to available system RAM	secure processor	enterprise-level	●	confidentiality
PMP-based TEE ^β	RISC-V (2017)	Xilinx Artix-7 SiFive E31, U54	multi-enclaves	changeable.	changeable.	-	mobile-level desktop-level	○	confidentiality integrity
Nitro Enclaves	AWS ^γ (2020)	Nitro Cards & Security Chip (EC2)	multi-enclaves	changeable.	up to available instance RAM	remote AWS service	desktop-level	●	confidentiality integrity
EdgeLock Secure Enclave	NXP (2021)	i.MX 8ULP(-CS) i.MX 9	single secure world	small.	changeable typically small	remote Azure Sphere	lot-level	○	confidentiality integrity
Confidential Compute Architecture (CCA)	Arm (2022)	Arm v9-A	multi-realms	changeable.	up to available system RAM	remote	mobile-level desktop-level	●	confidentiality integrity

^α Full TCB size is implementation-specific, and the reported TCB size here is the best case supported by hardware (bare-metal potential TCB). ^β Most difficult for fitting ML based on prior works. ^γ Strictly speaking, PMP (physical memory protection) is not a TEE but a hardware-based memory isolation feature that enables several frameworks to architect different TEEs on RISC-V. ^δ Nitro adds new infrastructure but still builds atop existing CPUs so strictly not an ISA.

Application Unchanged: ○ Major changes needed; ● Light changes needed; ● No change needed.

Table 2. Open Source TEE Frameworks

Open Source TEE Framework	Leading	Supported TEEs	On Top Of	Main Functions	Attestation	Languages	ML Examples
Intel SGX SDK [112]	Intel	SGX	firmware & driver	API and libraries for SGX app	●	C/C++	●
OP-TEE [113]	Linaro	TrustZone	firmware & driver	API and libraries for TZ app	○	C/C++	●
MultiZone [114]	HEX-five	RISC-V	firmware & driver	API and libraries for RISC-V Enclave app	●	C/C++	○
Keystone [115]	UC Berkeley	RISC-V	firmware & driver	API and libraries for RISC-V Enclave app	●	C/C++	●
Open Enclave SDK [116]	Microsoft	SGX, TrustZone	SGX SDK & OP-TEE	generalized API for TEE app	●	C/C++	●
Asylo [117]	Google	SGX	SGX SDK	generalized API for TEE app	●	C/C++	●
Tealache [118]	Apache	SGX	SGX SDK	API for Rust programming	●	Rust	●
Gramine [119]	Invisible Things Lab/Intel	SGX	SGX SDK	lightweight library OS to host app	●	Native executable*	●
Ocdm [120]	Tsinghua/ Ant Group	SGX	SGX SDK	lightweight library OS to host app	●	Native executable*	●
SCONE [121]	Scoutain	SGX	container & driver	TEE-based container image generation	●	Docker image*	●
Enarx [122]	Red Hat	SGX, SEV, CCA (upcoming)	firmware & driver	WebAssembly sandbox	●	WASM binary*	○
HyperEnclave [123]	Multiple inst	SGX, SEV	RustMonitor hypervisor	cross-platform enclave	●	multiple	○
Veracruz [124]	Arm	SGX, TrustZone, Nitro Enclaves, CCA (upcoming)	SGX SDK & OP-TEE Nitro Enclaves SDK	proxy attestation, policy WebAssembly sandbox	●	WASM binary*	●

Attestation: ○ Not available; ● Self-configuration required; ● Work off-the-shelf *Most modern languages supported by compiling as a target executable/binary. ML examples: ○ Not exist; ● Exists However, One non-exist example can be developed using an existing ML framework with corresponding languages.

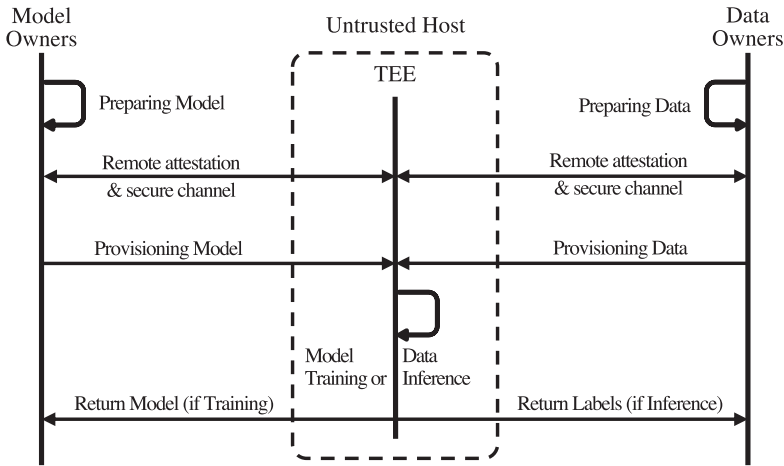


Fig. 3. Overview flow of Confidential Computing, which utilizes the untrusted host’s Trusted Execution Environment to protect the model and data in machine learning.

4 Confidential Computing Solution for Machine Learning

4.1 Overview

In Confidential Computing-assisted ML, data/model owners need to secretly provision their data/-model to the untrusted host’s TEE (see Figure 3). This process follows the procedure of how Confidential Computing works in general. Specifically, after preparation of the model and/or data, the owners first perform remote attestation to assure the integrity of the remote TEE and then establish secure communication channels to the TEE. Afterward, data/models are provisioned to the TEE, where model training or inference will be performed. Then, the produced results will be returned, i.e., a trained model will be transmitted out in training, or the data label will be returned back to the user in inference. Note that, in practice, the host, model owners, data owners, and result receivers could be distinct entities, or some may not exist, depending on use cases. We do not consider the case that one entity is “self-supplied” and does not need to interact with another entity, as there is no trust issue and the need for Confidential Computing. We follow the same logic when defining the paradigm of ML (e.g., centralized and distributed ML) and further classify these cases according to the nature of the host: Server or Client (see Figure 4).

Server-side ML protection. A server-like host aims to provide ML service to its customers. Based on the specific ML functionalities, the host can provide (i) **Inference as a Service (IaaS)**, e.g., References [32, 156, 157] or (ii) **Training as a Service (TaaS)**, e.g., References [31, 158]. In both cases, schemes should enable the protection of *data privacy*. That is, data have to be secretly provisioned to the untrusted server’s TEE no matter for inference or training. The private information contained in the model³ or its *intellectual property* also requires protection if it is owned by another entity (e.g., external model owners/providers), not the server. In this case, the model will be required to secretly provision into the TEE as well. However, the ownership of the ML model can belong to not only external owners but also to the untrusted server itself. The latter case will only exist for ML inference, as performing inference does not require changing the model parameters.

³Note that we do not use “model privacy” to avoid confusion, as the private information from the ML model is basically about the data used to train it, i.e., in a form of data privacy. The effort of building a model from these data is more properly called intellectual property.

① **Server-side ML:**

Protecting Data and/or Model

- Training as a Service
- Inference as a Service

② **Client-side ML: Protecting Model**

- On-device Inference
- On-device Personalization
- Federated Learning

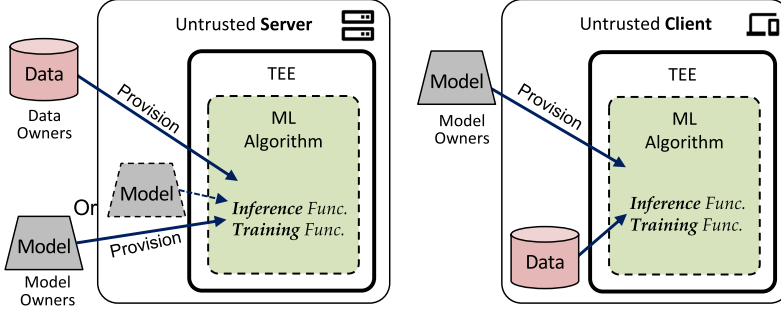


Fig. 4. Server-side ML (left) and client-side ML (right) protection using Trusted Execution Environments. Note that, in relation to the ML paradigm defined in Section 2.2, most centralized ML is server-side, and most distributed ML is client-side, depending on how the server and the client are defined under specific ML scenarios.

In contrast, training leads to updates of the model parameters on data, which consequently leak private information about the data (i.e., data privacy).

Client-side ML protection. A client-like host usually acts as the downstream user of a server, and it performs ML based on the ML framework/algorithm and models provided by the server. Specifically, common use cases include (i) **On-device Inference (ODI)**, e.g., Reference [159], the client conducts predictions on its own data with another individual's model that is downloaded and deployed on the device; (ii) **On-device Personalization (ODP)**, e.g., Reference [160], the client personalizes the existing model based on its data for later inference; and (iii) federated learning (**IaaS**), e.g., Reference [34], the client trains the model to further contribute a global model owned by another party (e.g., server). In all these cases, the client owns the data; thus, this client's TEE does not need to hide these local data from itself. However, the model is considered confidential, as it could be *intellectual property*, e.g., the model costs its owner or the server significant effort to train. More importantly, e.g., the model itself can leak private information about its previous trainers (server or other clients), as discussed in Section 2.3. The TEE of one client then dedicates to providing a trusted environment to respect other clients' data privacy.

4.2 Key Challenges of Adapting Confidential Computing

Adapting Confidential Computing to ML is not trivial work. The key challenges lie in the feasibility and effectiveness of utilizing TEEs for various ML services due to TEEs' limitations.

Constrained execution environments. First, TEEs have constrained resources for reducing the size of TCB. As given in Table 1, the commonly used hardware TEE, SGX, provides 128 MB secure memory, while TrustZone-based TEEs, such as OPTEE or trusty, by default provide up to 16~64 MiB secure memory. Note that the SGX 2nd [161] extensions allow additional flexibility in runtime management of enclave resources (e.g., adding memory to an enclave after the enclave is built and running) and multithreaded execution within an enclave. Further, the SGX 3rd and 4th Generation from Intel® Xeon® Scalable Processors can support up to 512 GB, hence, 1 TB of EPC on a two-socket platform [162–164], mostly due to using **Advanced Encryption Standard-XEX Tweakable Block Cipher with Ciphertext Stealing (AES-XTS)** instead of previous **Memory**

Encryption Engine (MEE), to scale EPC size. Though these CPUs are not widely available to the public, some companies are utilizing these more scalable SGX features combined with high-performant AI enhancements like Intel **AMX (Advanced Matrix Extensions)** to gain orders of magnitude performance boost for SGX-assisted federated training or inference.⁴

Although techniques using page swapping can increase available memory, it leads to significant overhead (e.g., 100~1,000×) when running ML [32, 167]. Depending on the TEE software stack and architecture, the size and flexibility of protected memory vary. However, such memory sizes are still highly limited compared to the memory consumed by current ML algorithms, which can reach hundreds of MBs or GBs.

In addition, processor capabilities are limited when running in the TEE mode. For example, privilege instructions are not allowed inside SGX enclaves, which makes supporting widely used functionalities (e.g., filesystem or multi-threading) difficult or even impossible. For instance, the SGX multi-threading model relies on an untrusted OS for thread scheduling that leads the system to be vulnerable to synchronization-based attacks (e.g., AsyncShock [168]). Similarly, making the secure world over-privileged in TrustZone architecture caused mobile vendors to only provide limited TEE services and much-restricted TEE usage. Indeed, increasing computational resources is always preferable for ML developers, but it can reduce the reliability and practicality of TEE-based solutions.

SDK supports. A similar challenge arises due to the limits of trusted resources. Modern ML framework usually requires numerous libraries and cross-compilation to support high-performant data loading and computations (e.g., matrix multiplication). However, most TEEs provide basic low-level SDKs. Even though open-source TEE frameworks have been developed to support sandbox/-container (see Table 2), it is still hard to port all ML dependencies and necessary libraries into TEEs. For example, WebAssembly supports compiling binaries from many modern languages, e.g., Rust, which could have available ML libraries, such as autograd [169], but they have much-limited functionality compared to TensorFlow or PyTorch [170]. OS-based containers can provide richer libraries, but still, the lightweight version OS cannot have a rich environment same as the normal OSs like Linux. Porting a normal OS to TEEs is also infeasible (and overkill in many use cases), because it either exceeds the TEEs' size limits or leads to a huge TCB that destroys TEEs' security benefits.

Privacy protection effectiveness. Furthermore, even if one manages to deploy ML inside TEEs (for training/inference), the untrusted host itself cannot access the ML computations anymore. Running inside TEEs does not automatically disable all potential privacy leakages, as typically the produced results will need to be passed to outside of the TEE. Specifically, the other parts of the ML pipeline still leak private information about the hidden process inside the TEE through various attack vectors. For instance, predicted data labels or trained models usually need to be transmitted out of the TEE's trust boundary to a broader area, e.g., IPC or file system, that attackers can leverage to exploit membership information or even the original data as shown in References [17, 20, 160, 171–173]; thus, one may need to add another layer for stronger privacy protection, e.g., using techniques in research [174, 175]. Also, one particular protection strategy could have inconsistent protection effectiveness for other ML scenarios and different types of defined privacy [176–178]. As one example, while hiding model parameters (along with gradient updates and activations) inside TEEs can defend against DRAs, it has very low efficiency for MIAs, as most

⁴For example, Demetis Medical Technology Co. Ltd. uses scalable SGX and oneMKL to secure its AI algorithms and IP for medical devices (oneMKL and oneDNN enhance the performance and compatibility of Intel processors and deep learning frameworks) [165]. Another example is BigDL PPML, a solution that protects the big data and AI pipeline, created by Intel and Alibaba Cloud DataTrust [166].

membership information can leak from the model's outputs, i.e., prediction results transmitted out of the TEE [93, 174, 179]. Therefore, for one specific TEE-assisted use case, the target privacy should be defined clearly, and the achieved protection efficiency also requires explicit analyses.

4.3 Existing Solutions of Confidential ML

Previous research has been dedicated to achieving ML with TEEs, and some aim to overcome the above challenges. We summarize previous research in Table 3. We can see the trend that much early-stage work mostly focuses on centralized ML/server-side ML until the year around 2020 several works started researching distributed ML/client-side ML. Most recent research is on enabling TEE functionalities on AI accelerators (e.g., GPUs, NPUs, IPUs, etc.) for secure or private ML. Note that these works on accelerators make great effort on the partitioning framework, system design, and even cryptography to enable TEEs, but the main goal is to support ML; therefore, we still include them here instead of the previous section on TEE frameworks.

In addition, when there is a smaller number of mistrusting entities, guaranteeing their confidentiality/integrity can be consequently easier. For example, in TaaS or IaaS, where only one client uses the server's TEE at one time, we need protection of only the user's properties and, therefore, can protect against all confidentiality-related attacks. In an FL case where multiple clients participate, defending against all these attacks can be very difficult, because the final trained model will highly probably be passed to participating clients, which consequently leaks information.

We also note that these different solutions are still measured in similar metric vectors such as system performance overload on the baseline. This can be execution time, memory resources, or sometimes power consumption on edge devices. The most commonly used one is execution time as a direct measure of the processor's performance. Nevertheless, the chosen baseline can vary a lot, from native CPU execution, to common TEE such as SGX, to GPU execution (as shown in Table 3). The effectiveness of the TEE's protection usually is not directly measured by standardized metrics; instead, the developers give "security analyses" on their system to logically compare with native systems or other protection mechanisms.

In the following subsections, we present existing solutions with their features in detail.

Complete ML training/inference inside TEEs. The most straightforward approach is to deploy a complete ML training/inference process inside TEEs. In such a case, the maximum capability of the ML task is strictly limited to the TEE's space and computing constraints. Much early-stage work, e.g., References [29, 31], implemented and tested on small-scale ML algorithms so probably can fit the complete inference or even training process into TEEs. In addition, such a full deployment is also tested in later research under FL scenarios based on the assumption that ML on clients' edge devices is usually light, while remote attestation can be further adapted to manage the agreement among all clients [191]. However, when the required secure memory of training a larger ML model exceeds the TEE's size, fitting it as a whole becomes infeasible. Then, one strategy is to use memory space efficiently—trading off the total number of the model's layers and the number of neurons—and to maximize the efficiency of every "bit" of TEEs' secure memory for ML computation. Specifically, we list several practical approaches as follows: (i) Conducting inference instead of training, e.g., References [32, 159, 181]. Training consumes much more computational resources because of backward propagation (e.g., memory used to save model gradients and intermediate activations). (ii) Choosing a small batch size. A large batch size leads to large memory usage, because every sample in this batch produces its own activations for all model layers. (iii) Balancing the feature extractor (e.g., convolutional layers) and the classifier (e.g., fully connected layers) [34]. A properly designed feature extractor can decrease the feature dimension but still capture key features, enabling a compact classifier to achieve good performance.

Table 3. Previous Research that Uses Trusted Execution Environments to Guarantee the Confidentiality/Integrity of Machine Learning

Work & Year	Confid. Computing			Machine Learning				Advanced Features
	TEE Type	SDK/Partitioning Framework	DL Library	Train-ing	Paradigm, Loc., & No. of entities ^a	Protect Against	Overhead (execution time) ^b	
Obliv. MP'16 [29]	SGX	Intel SGX SDK	fast CNN	●	TaaS, Server, Multiple	C(DR, AI)	1% – 3% on Native	Data-obliviousness
Chiron'18 [31]	SGX	Intel SGX SDK	Theano	●	TaaS, Server, Single	C(AI), I(MP)	4% – 20% on Native	Multi-enclaves, Ryoan VM
PRIVADO'18 [156]	SGX	Intel SGX SDK	ONNX	○	IaaS, Server, Multiple	C(DR, AI)	~ 17.18% on Native	Compiler, data-obliv.
Myelin'18 [158]	SGX	Intel SGX SDK	TVM	●	TaaS, Server, Multiple	C(DR, AI)	4.7% – 9.2% on Native	DP for data obliviousness
Slalom'18 [30]	SGX	Intel SGX SDK	Eigen	○	IaaS, Server, Single	C(AI), I(MP)	SU. 4 – 20% on SGX	Freivald's algorithm + GPU
Graviton'18 [180]	GPU-TEE	CUDA RT driver	Caffe	●	TaaS, Server, Single	C(AI), I(MP)	17% – 33% on GPUs	Secure CUDA for TEE on GPUs
Occlusionary'19 [32]	SGX	Intel SGX SDK	Caffe	○	IaaS, Server, Single	C(AI), I(MP)	~ 72% on Native	On-demand loading, Channel partition
TensorSCONE [167]	SGX	Intel SGX SDK	TF	●	TaaS, Server, Single	C(AI), I(MP)	~ 3× on Native	Docker supported, Compiler
YerbaBuena'19 [181]	SGX	Intel SGX SDK	Darknet	○	IaaS, Server, Single	C(DR)	up to 7.5% on Native	Layer-wise partition, Privacy measure
Origami'19 [182]	SGX	Intel SGX SDK	Eigen	○	IaaS, Server, Single	C(DR)	SU. 2 – 15× on CPUs	Model partitioning, GAN attacks
OMG'20 [159]	TrustZone	SANCTUARY	TFLM	○	ODI, Client, Single	C(MS)	2.11× on Native	-
DarknetTZ'20 [160]	TrustZone	OP-TEE	Darknet	●	ODP, Client, Single	C(MI)	3% – 10% on Native	Layer-wise partition, Privacy measure
TrustFL'20 [183]	SGX	Intel SGX SDK	TF	●	ODP, Client, Single	I(MP)	~ 2× on Native	GPU-outsourcing, Random sampling
Tekline'20 [184]	GPU-TEE	ROCm & CUDA	MXNet	●	TaaS, Server, Single	C(AI), I(MP)	10% – 41% on GPUs	Timing attacks, data-obliv.
Serdab'20 [185]	SGX	Intel SGX SDK	TFlite	○	IaaS/ODI, S+C, Single	C(AI), I(MP)	SU. 4.7× on one SGX	Multiple TEEs, Resource management
HybirdTEE'20 [186]	SGX + TZ	OP-TEE + SIGMA	Darknet	○	IaaS/ODI, S+C, Single	C(AI), I(MP)	SU. 1 – 6× on Client	Hybrid TEEs, Partitioned ML
AegisDNN'21 [187]	SGX	Intel SGX SDK	TF, PyTorch	○	IaaS, Server, Single	I(MP)	Realtime service	Safety Profiling of Layers
MLCapsule'21 [188]	SGX	Intel SGX SDK	Eigen	○	ODI, Client, Single	C(MS)	55% – 116% on Native	-
Trusted-NN'21 [189]	TrustZone	OP-TEE	Not reported	○	ODI, Client, Single	C(MS)	22.8% on Native	Weights & Feature-map partition
Mem-Eff'21 [190]	SGX	Azure CC (VM)	Darknet	○	ODI, Client, Single	C(MS)	9% – 100% on Native	Channel & Y-plane partition
PPFL'21 [34]	SGX + TZ	OE + OP-TEE	Darknet	●	FL, S+C, Multiple	C(AI)	~ 15% on Native	Layerwise training, Privacy measure
SecFL'21 [191]	SGX	SCONE	TF	●	FL, Client, Multiple	C(AI)	Not reported	Remote attestation
Goten'21 [192]	SGX	Intel SGX SDK	Eigen	●	TaaS, Server, Single	C(AI), I(MP)	SU. 6.84× on SGX	GPU-outsourcing, Non-colluding servers
Citadel'21 [193]	SGX	SCONE	TF	●	TaaS, Server, Multiple	C(DR,AIMS)	9% – 73% on Native	Multi enclaves for training
SecDeep'21 [194]	TrustZone	ATF + OP-TEE	ARM NN DL	○	ODI, Client, Single	C(MS)	16×–172× on Native	GPUs, Controlling kernel page table
Fair. Audit'22 [195]	SGX	SGX-LKL-OE	PyTorch	○	IaaS, Server, Single	C(AI), I(MP)	~ 30% on Native	Fairness audit, Modler + regulator
GuardNN'22 [196]	FPGA-based	(Sim.) CHaiDNN	SCALE-Sim	●	TaaS, Server, Single	C(AI), I(MP)	up to 1.07% on Native	-
SOTER'22 [197]	SGX	Gramine	PyTorch	○	ODI, Client, Single	C(MS), I(MP)	up to 1.27× on GPUs	Oblivious fingerprinting for integrity
StrongBox'22 [198]	TrustZone	Midgard + ATF	ARM NN DL	○	ODI, Client, Single	C(MS)	2.8%–19.7% on GPUs	Fine-grained memory protection policy
Dash'23 [199]	SGX	Intel SGX SDK	ONNX	○	IaaS, Server, Single	C(AI), I(MP)	SU. 5–140× on CPUs	Garbled circuits, MPC, GPUs
ITX'23 [200]	IPU-TEE	IPU/CCU Stack	TF	●	TaaS, Server, Single	C(AI), I(MP)	<5% on IPU	remote attestation, AI accelerator
Honeycomb'23 [201]	SEV-SNP	SVSM & SM & HIP	PyTorch, etc.	●	TaaS, Server, Single	C(AI), I(MP)	~ 2% on GPUs	Software-based, Small TCB
T-Slices'23 [202]	TrustZone	OP-TEE	Darknet	○	ODI, Client, Single	C(MS)	SU. 29% on DarknetTZ	Dynamic fragmentation of DNNs
AvaGPU'23 [203]	TrustZone	OP-TEE + CUDA	TensorRT	○	ODI, Client, Single	C(MS), I(MP)	~ 15.87% on GPUs	real-time CPS system, Small TCB
SecureLoop'23 [204]	TEE-like DNN Accelerators in general			●	N/A	N/A	SU. ~ 33.2% on GPUs	Design space, Cryptographic engines
Deluminator'23 [205]	SGX & TrustZone			○	ODI, Client, Single	C(AI), I(MP)	~ 7% – 29%	Information flow tracking, Auditing
PoUL'24 [206]	SGX	Intel SGX SDK	Intel DNNL	●	TaaS, Server, Single	C(AI), I(MP)	N/A	Unlearning & Proof of unlearning
CAGE'24 [207]	CCA	RMM & SM	Not reported	○	IaaS/ODI, S/C, Single	C(AI), I(MP)	1.2% ~ 7.6% on GPUs	Unified-memory GPUs, Small TCB

^a Paradigm: Categories following Figure 4; Loc.: Locations of used TEEs; No. of entities: Number of mistrusting participating entities to be protected.

^b The reported overhead is given to show general performance and *cannot* be cross-compared due to measuring under different experimental settings.

SU. refers to speed-up, instead of overhead. Training: ○ Not available; ● Available

Protect Against: C(DR) = Confidentiality - Data Reconstruction; C(AI) = Confidentiality - Attribute Inference; C(MI) = Confidentiality - Membership Inference;

C(MS) = Confidentiality - Model Stealing, I(MP) = Integrity - Model Poisoning; I(DP) = Integrity - Data Poisoning

Note that the difficulties of achieving one protection aim also depend on the number of mistrusting participated entities to be protected; here, we refer to the state that all participating entities' confidentiality/integrity are guaranteed if filled.

Partitioned execution. To avoid exceeding the maximum secure computational resource as well as memory swapping, one approach is leveraging efficient compartmentalization (or partitioned execution) to actively optimize memory usage in the ML process. Figure 5 shows two types of partitioned execution: (i) Layer-based partition [34, 160, 179, 208, 209], which works for models with layer architecture in general, and (ii) Feature map-based partition [189, 190, 210], which specifically aims to convolutional layers due to the high memory cost of this type of layer. Among all layer candidates, whether to keep the first part or the last part inside TEEs (cases ① and ②) is one key choice to be confirmed, which can depend on the security/privacy protection goal. For example, protecting the first layers guarantees the privacy of *original* information, while protecting the last layers guarantees the privacy of *membership* information [93, 171, 177]. Rolling on

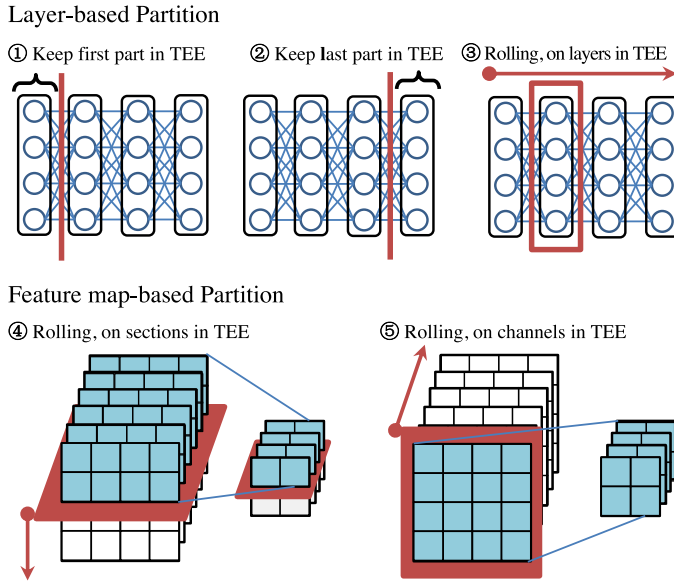


Fig. 5. Partition on the ML process (shown with model architectures) to protect it or a part of it inside TEEs.

layers inside TEEs (case ③) naturally works for ML inference, because one input propagates forward throughout layers and never goes back; it works in the way that the next layers are loaded into a TEE after previous layer's computation executes finished in the TEE. Rolling on feature maps (cases ④ and ⑤) reduce the realtime memory usage by applying **GEMM (General matrix multiply)** and **img2col (Image to column format transform)** functions on divided sections and channels of feature maps. However, this disables parallelization features and, therefore, can increase computation time.

Furthermore, although partitioned execution may be more necessary for training due to the more resources required in the backward propagation, partitioned execution for training could lead to higher overhead than inference, specifically in ①, ②, and ③. Evaluations in previous work [34, 160] indicated an increase from 10% to 20% approximately in their settings). The overhead is caused by a higher volume of the buffer being transmitted between the TEE and normal OS. Note that to enjoy higher flexibility to support generic ML models, the partition should better not change the model architecture (unlike the model partition in the design of AlexNet [211]).

TEE-assisted accelerators. In addition to optimizing the limited TEE-protected resources, one can also extend the computation power and resources to accelerators (e.g., GPUs, NPUs, TPUs, and FPGAs) [180, 192, 212–214]. However, most accelerators are considered untrusted and do not support TEE-assisted functionalities. To address this issue, one way is to use CPU-TEE (e.g., SGX) to coordinate the computation delegation and to establish a secret sharing protocol between the CPU-TEE and untrusted accelerators. In such a way, accelerators are able to perform executions on a “nonsensitive version” of the TEE computation, e.g., masked by one-time pads [30, 192]. This can also accelerate the ML's linear computation, i.e., matrix multiplication. There are also TEE-assisted record-and-replay-based solutions, which are more suitable for resource-constrained and mobile GPUs. As an example, CODY [214] allows the full computation to be done inside GPU and uses TEEs like TrustZone for secure logging and inference monitoring. However, due to the complexity of heterogeneous SoC architectures, software-only secure channels between TEEs and

accelerators without proper hardware **root-of-trust (RoT)** still leave a large attack surface and consequently weaken TEE security guarantees in turn.

Another fundamental approach is to extend accelerator hardware, systems stack (e.g., the driver), and API support for enabling accelerator with more hardware compartmentalization and TEE-assisted features. Previous works [180, 184] achieve GPU-TEE by relatively small hardware modification (unmodified GPU/CPU core but modified peripherals like PCI) and the necessary system software (e.g., runtime, hypervisor, or drivers, etc.). These solutions could add up to 41% overhead on native GPUs. Similarly, SGX-FPGA [213] builds a secure hardware isolation path between CPU and FPGA to protect the sensitive data in both components and in interactions. It extends the capability of the original SGX to a counterpart FPGA enclave by leveraging the FPGA's **physical unclonable function (PUF)** to achieve the hardware RoT on the heterogeneous SoC. **ITX (IPU Trusted Extensions)** [200] follows a similar logic by extending a set of hardware and software on Graphcore **Intelligence Processing Unit (IPU)**, a type of SOTA custom AI accelerators, with encrypted data stream through PCIe. Evaluation of the implementations showed only <5% performance overhead on native IPUs. This approach is also integrated into some recent NVIDIA GPUs, such as H100 Tensor Core GPU Architecture [215].

More specifically, NVIDIA introduced **Multi-Instance GPUs (MIG)** like Hopper [216], which enables each GPU to be partitioned into several smaller, fully isolated instances with their own memory, cache, and compute cores. This HW-assisted compartmentalization approach allows more flexible implementation of trusted computing domains with trusted IO paths. The Hopper architecture further enhances MIG by supporting multi-tenant, multi-user virtualized domains across up to seven GPU instances, securely isolating each instance with Confidential Computing at the hardware and hypervisor level. For instance, it provides dedicated video decoders for each MIG instance to deliver secure, high-throughput **intelligent video analytics (IVA)** on shared infrastructure, as well as concurrent MIG profiling, for secure optimization of resource allocation.

Hence, despite the current limitations, Confidential Computing with more flexible features would be supported on future accelerators. At the same time, ML frameworks and systems must still be prepared to utilize these features properly, which can not be merely resolved from the userspace perspective in an efficient and high-performant way due to the limitations of current systems abstractions that are not designed with such hardware-security features in mind. For instance, previous work shows how extending the underlying OS and system stacks for enabling more extensible and TEE-aware compartmentalization, called dispersed compartments [217], could facilitate properly utilizing such hardware-based compartmentalization features.

Attack-based privacy measurement. The TEE protection needs measurements to show its effectiveness. Currently, the main approach for measuring TEE security is through red-teaming and performing various attack vectors directly. Whether to and which attack to perform mostly depends on the threat model defined for the TEE, which could be inadequate for different ML use cases. As one example, regarding the original input protection with TEEs, one performs attacks to reconstruct input data [179] while there is no need to conduct membership inference attacks. By contrast, TEE protection on membership information does not take care of other private information and only be measured with membership inference attacks [160]. However, due to the poor definition of various types of privacy, the attack-based analysis lacks a theoretical foundation on privacy leakage, which sequentially makes the TEE-based privacy guarantee less trustworthy.

5 Confidential Computing Helping Machine Learning Integrity

Intuitively, other than privacy guarantee, deploying ML training/inference process into TEEs could avoid malicious modifications to this process and therefore ensure integrity. However, in practice,

as the other parts of the ML pipeline still can be breached, protecting only the training/inference stage does not always guarantee integrity. For instance, the training happening outside can be modified maliciously in partitioned execution while tackling space-constrained problems. In this section, we present the key challenges in enabling ML integrity using TEEs and the solutions that exist or can be adapted.

5.1 Key Challenges in Enabling ML Integrity

Large attack surface. There is a large attack surface to breach integrity; however, one cannot expect to deploy the complete ML lifecycle inside TEEs to achieve an integrity guarantee due to the TEEs' resource constraints. These limitations for achieving a proper integrity guarantee are even more severe compared with the issue we met when guaranteeing confidentiality. Specifically, while an honest-but-curious (i.e., confidentiality-related) adversary has the goal to explore specific private information, the integrity-related adversary can be more diverse—it aims at not only actively changing the ML process but also even breaking the process by covertly changing one bit in updated weights/gradients. Indeed, to disclose confidential information, adversaries usually explore the target data/model, and contained information in the model may also gradually decrease along with more aggregated information in training. However, to breach integrity, one just needs to change one step in the training process (even in one bit) among participants (e.g., Byzantine attacks [14] or synchronization attacks [87]). This malicious change can also be triggered at other stages, e.g., data preparation.

Uncontrolled input/output space. While deploying ML training/inference inside TEEs avoids unauthorized direct changes in the produced result/models, the upstream pipeline, i.e., data preparation, or input space, is uncontrolled. Editing the inputs and their labels can affect the later ML process (e.g., called “dirty” inputs/labels), which is also the practice of performing some attacks, e.g., poisoning attacks or adversarial example attacks. Adversarial examples [22, 85] that add calibrated noises to the input can seriously perturb the integrity of the input space due to the hardness of detecting the noisy perturbation. In addition to that, the downstream pipeline can be edited maliciously. For example, after the ML algorithm inside the TEE makes a prediction and transmits this result out, a server-side adversary fakes the result, which is also hard to detect on one result receiver side.

Low interpretability of ML. One major reason for the difficulty of detecting integrity perturbations is the low interpretability of current ML/DL [218]. Specifically, current practitioners train ML models using SGD, i.e., weights of neurons are updated iteratively to find the optimal solution. There is still no comprehensive theoretical framework to fully interpret the internal steps of ML training. Therefore, one cannot determine whether the weight value of a neuron is uncompromised or not without *re-running* the ML process using the same setting. With such low interpretability, malicious changes such as the adversarial perturbation in inputs and the changes in model training are hard to trace. Algorithm-based approaches have been proposed to alleviate this situation, e.g., by removing outliers that are considered as unreasonable bits [14], by further training the model on adversarial inputs [219] or by cryptography protocols [220]. However, utilizing a system-based isolation technique (i.e., TEEs) to deal with the interpretability issue is not straightforward. One may still need to find a way to identify and ensure key elements of the ML pipeline are trustworthy using TEEs, i.e., rerunning and verifying sensitive parts of the ML.

5.2 Existing/Adoptable Solutions

Assurance mechanisms. To assure the integrity of ML training/inference, one way is to rerun the complete or a part of the process inside TEEs. For example, the heaviest computation in ML is the

matrix multiplication, which is preferable to be outsourced to distrusted GPUs for acceleration. To verify whether this matrix multiplication is honestly performed, Reference [30]’s work uses Freivald’s algorithm [221], which utilizes randomization to reduce time complexity from $O(n^{2.373})$ (best-known matrix multiplication algorithm) to $O(kn^2)$ with a probability of failure less than 2^{-k} . Besides, based on the nature of iterative training of ML, one can also sample and verify only a fraction of the training process in TEEs to reduce the overhead [183, 222]. This will require backups for every several training iterations to reach a checkpoint faster, where Merkle hash tree-based method [223] can be used to reduce the storage overhead. With such an assurance scheme, Reference [183] demonstrates that adversaries only have a 1% probability of successfully cheating even despite that they honestly completed 90% of training rounds. Furthermore, there could exist more efficient ways of sampling or verification based on watermarking [224], but how such an assurance guarantees the integrity of other stages in the ML pipeline deserves more investigation.

Input/output space control. The integrity of input/output in the ML pipeline needs to be assured considering that the input/output space is far more uncontrollable than the training/inference process. One way that is generally used for data assurance and can also be adopted in ML input control is to require a digital signature for underlying data generation. This can be done by hashing the generated sensor data (inside TEEs, for example) to produce hash values. After that, sensor data are authenticated using these hash values [225–227]. At a later stage, the digital signature would also work in a similar way for remote attestation when outputting prediction results, considering that the result receiver trusts the TEE’s behavior. Nevertheless, it becomes harder especially for supervised learning, because the data generation usually involves human annotations that are out of the control of the digital signature.

Even in semi-supervised/unsupervised learning, it is possible to generate adversarial data without breaching sensors/TEEs, e.g., by changing physical surroundings or doing abnormal behaviors that the ML does not expect to learn. Therefore, the exploration of how to assure the integrity of the input space using TEEs is still a research question. One example can be adopting detection mechanisms (e.g., References [228–230] to identify fake/dirty inputs inside TEEs, which deserves further investigation. Previous work also introduced system-wide TEE-assisted information flow control, particularly over peripherals and OS services to detect and mitigate such unauthorized IO-based attack vectors [231]. However, these techniques require fundamental changes in system software including commodity OSs.

6 Limitations of Existing TEE Systems

Despite the benefits of TEEs/enclaves, the right abstractions for securely and efficiently utilizing them are still not clear, particularly when combining various TEE solutions (e.g., for multi-platform use cases) or integrating with existing systems. Below, we further detail the limitations of existing TEE systems that obstruct their use in ML.

6.1 Widening the Attack Surface

New features lead to new attack vectors. TEEs require rich functionalities and performance features to support running modern ML programs. However, merely adding hardware features without a comprehensive security model and mechanism with existing system software and privilege boundaries (e.g., sharing data or resources) can even increase the system’s complexity and open new attack vectors, particularly in TEE architectures, where the untrusted host OS or hypervisor is responsible for managing TEE/enclave system resources. For instance, recent work such as SGXRacer [232] shows more than 1,000 new vulnerabilities through controlling/misusing kernel scheduler and SGX shared (i.e., the racing) variables. Also, recent security evaluations of Intel TDX

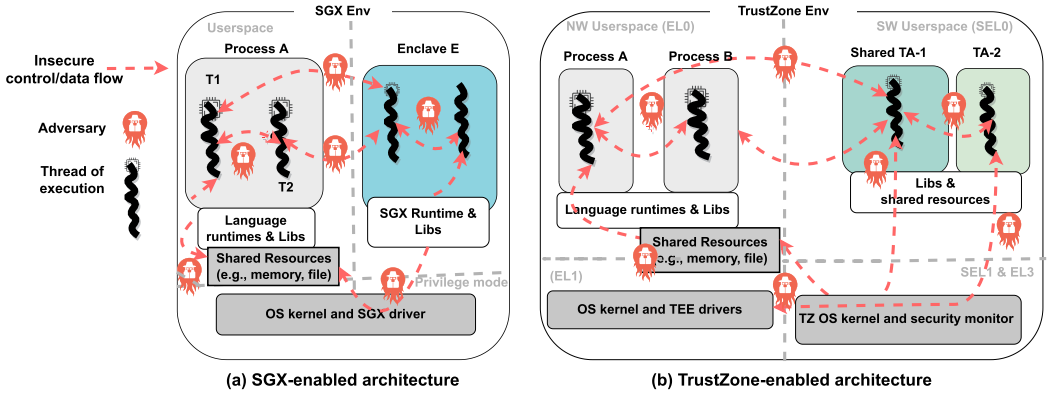


Fig. 6. Unsafe or malicious data/control flows in TEE systems cause various attack vectors within and across trust boundaries in TEE systems. An adversary could exploit vulnerabilities within almost all layers of abstractions and software stacks, as shown in previous research. This figure simplifies some of these unsafe control and data flows for launching an attack or propagating different vulnerabilities to compromise the confidentiality or integrity guarantees of a TZ-assisted or SGX-assisted system.

have revealed various vulnerabilities in different layers of the TDX software stack [233], including its attestation services, which also include SGX-based quoting enclave, during TD live migration (a new feature), and TD binding[234], which may allow information leak and escalation of privilege (e.g., see CVE-2023-45745 and CVE-2023-47855).

As another example, ARM TrustZone introduced hardware support for secure and non-secure worlds by adding several privilege levels in each world. However, the architecture limitations, such as the insecure sharing mechanism and fixed/one-way security model [235], combined with insecure software designs, lead to new attack vectors, particularly when sharing resources or exchanging data/control between different privilege layers [107, 108]. Figure 6(b) demonstrates some of these attack vectors in a TrustZone-assisted environment.

Previous attacks, such as Boomerang [107] and HPE (Horizontal Privilege Escalation) [236], also show how the semantic gap between heterogeneous compartments (e.g., userspace processes and TrustZone TAs (Trusted Apps)) leads to severe privilege escalation threats. Boomerang exploits a confused deputy vulnerability inside a TA via the shared memory between the two worlds and takes advantage of secure world privilege to make the host kernel memory accessible. Figure 6(b) shows simplified data/control flows of such attacks through compromised user space processes or libraries to a vulnerable TA via shared resources/RPCs and then through that compromised TA to the secure kernel for finally taking control of the system. Similarly, HPE attacks demonstrate that an adversary process could compromise another userspace process through a shared TA (e.g., a shared crypto TA) or insecure privilege management between two TAs. For example, in Figure 6(b), an adversary could misuse the TA_1 's shared persistent state and logs between process A and B to leak sensitive information from one process to another or propagate through TA_2 or other unauthorized sources.

Huge TCB caused by in-enclave LibOS. Following the in-enclave LibOS approach for porting unmodified yet complex applications into enclaves results in a huge TCB through porting all dependencies inside the enclave (and more recently inside the entire **virtual machine (VM)** via TDX or SEV features). Such designs force developers to run large code in a single enclave, resulting in inefficient and over-privileged enclaves. This leads to a wide range of new attacks due to exploiting in-enclave vulnerabilities [102, 168], insecure interactions with outside [102, 237, 238],

or misusing enclaves to escalate privileges [107, 236, 239]. Currently, there is no systematic way to detect or protect against these threats.

Vulnerable interactions with outside. None of the existing partitioning frameworks fully consider the complex attack surface originating from insecure interactions between the host hypervisor, OS, userspace processes, and enclaves. For instance, an untrusted/malicious hypervisor can inject virtual interrupt to affect AMD SEV-SNP and AMD SEV-ES (CVE-2024-25742), compromise guest memory integrity through improper address validation or memory safety issues (CVE-2023-20566 and CVE-2023-20519). As another example, Vicarte et al. [87] show how asynchronous poisoning attacks on TEE systems lead to changing the accuracy and integrity violation of in-enclave ML models. Civet uses dynamic taint-tracking to control the flow of objects on enclave interfaces but cannot help for proper privilege separation and against more complex attacks (e.g., HPE attacks [236]) and other languages. Moreover, they do not consider in-address space compartmentalization as the key issue of handling over-privileged enclaves. Sirius [231] shows that targeting these attacks requires fundamental changes to underlying OS and TEE systems. Note that developers sacrifice performance and rich functionality for the security benefits of TEEs; which could be a huge cost without actual benefit considering the insecure designs of existing TEE systems.

ML inside TEEs but still being attacked. Existing vulnerabilities of TEEs still leave the deployment of ML under some forms of attacks. For example, by observing the access pattern to TEEs, previous work in Privado [156] can disclose the target class information, i.e., classifying encrypted inputs with high accuracy (97% and 71% for MNIST and CIFAR10, respectively). Also, side-channel attacks on TEEs, such as timing, power, and electromagnetic analysis, could compromise the ML model privacy on some levels, e.g., reconstructing the model architecture and even model parameters [240–242]. Some of such attacks can be out of scope, because our threat model does not include side-channel attacks. However, this still reflects the current limitations of the existing TEE system, which deserve further investigations in future design (e.g., efficient oblivious operations for ML [29, 243]).

6.2 Programmability & Performance

The quality of ML solutions highly depends on the proper processing of an enormous amount of data. However, almost all TEE systems are not designed considering this essential requirement. Many TEEs lack minimal hardware resources for efficient ML programming on modern CPUs, as we explained earlier (Section 4). Currently, resolving these limitations causes significant additional performance overhead. As an example, MPTEE [244] proposes a mechanism for supporting more flexible memory management and dynamic permission enforcement inside enclave memory. It uses MPX's three bound registers to offer the six common memory permissions (i.e., RWX, RW, RX, R, X, non-permission) with about 35% overhead on average and 57% in some cases. Besides, Occlum [245] proposed an MPX-based SFI technique for supporting in-enclave multi-processing, which adds a 10× slowdown compared to Linux processes. SGXv2 adds special instructions (e.g., EMODPR and EMODPE), to facilitate some of these issues including dynamic memory management. However, the overhead is not clear yet due to the lack of hardware support, and SGXv1 is still the dominating version for a while.

These improvements are not sufficient for most ML use cases. First, current designs cannot be integrated with a wide range of accelerators such as GPUs, FPGAs, and TPUs. Despite recent efforts for supporting TEEs in these accelerators as described in Section 4.3, their systems software and ML framework designs and evaluations are in the early phases [180, 184, 213, 246]. Reference [247]'s SoK paper also shows why the existing TEEs are unsuitable for high-performance computing systems; even for new TEE architecture like CCA, improvements are still required to facilitate

its use in protecting the entire ML deployment pipeline [248]. Depending on the level of protection in different ML stages, the overhead and toolchain supports can be highly diverse. Also, none of the previous work offers scalable solutions with multiple accelerators, which deserve further investigation.

It is worth noting that (which is usually ignored) programming ML solutions with TEE systems has a high learning curve due to the huge differences between their architectures and programming logic. The complexity of the integration requires expertise in both fields that may need years to build for developers. Although proficient developers can build preliminary solutions, due to the fast changes in ML software stacks, it is still hard to maintain the pipeline; thus, the solution cannot be scaled to a larger community.

6.3 Heterogeneity & Migration

Heterogeneous SoC architectures enable a wide range of functionalities and are now available even for modern IoT/edge platforms [249]. Modern SoCs contain heterogeneous CPUs (e.g., a combination of ARM and RISC-V architectures) and peripherals. As a result, the system stack on such devices includes multiple OSs (e.g., Linux and FreeRTOS), hypervisors, and different TEEs. As a simple example, recent NXP's i.MX boards, simultaneously support different TrustZone implementations for mixed cortex-A and cortex-M cores (e.g., i.MX 8ULP or 8QuadXPlus). Secure hardware partitioning and sharing hardware resources within such ever-growing complexity is challenging. That is the reason why most systems software (e.g., Android [250]) does not expose heterogeneous TEEs to userspace applications despite hardware availability.

Moreover, heterogeneous TEEs could have different security models and resource constraints, which make the migration of TEE-based ML solutions from one platform to another much more challenging. For instance, remote attestation supports in TrustZone-based and SGX-based solutions are different as well as their system stacks (including the support for fuzzing, debugging, auditing, etc.). Naively, combining different security protocols does not lead to a secure solution. Also, as summarized in Figure 6, vulnerabilities can easily be introduced and propagated in various privilege boundaries (within the different or same address spaces). Detecting and protecting against such a complex attack surface is one of the primary obstacles to securely handling TEE migrations. Ad hoc approaches are neither practical nor scale well. We need a principled approach to deal with heterogeneous TEEs efficiently and consider future security threats [205]. A naive migration could widen or worsen the attack surface while reducing the performance of ML solutions in most cases [217].

7 Next Generation and Beyond

In this section, we present the potential approaches to tackle the above discussed issues, toward better privacy-preserving machine learning atop TEEs.

7.1 Strengthen Privacy Foundation in Confidential Computing

As one of TEE's primary protection goals, privacy needs to be strictly defined and preferably be defined with theoretical basics (e.g., information theory). While most privacy measures are empirically evaluated by conducting attacks,⁵ defining privacy theoretically is complicated due to the broad existence of attacks in ML; one rigorous definition can fail to explain other attacks.

Among candidates, *differential privacy* is a well-accepted definition of privacy. DP has been massively explored when applied to ML. Privacy audits using DP can obtain a lower bound but usually

⁵ Measuring privacy using attack success rates fails to explain and quantify privacy leakages and then leads to "Arms Race," where current protections need re-evaluation or re-design when new attacks happen.

require running the training/estimation algorithm hundreds of times. One promising work [251] explores the parallelism of adding or removing multiple training examples independently and reduces the privacy auditing into one training run. However, it still faces the same generalization problem. As one example, one can apply DP-training [252] to show the protection efficiency against DRAs that recover original training data from gradients computed on it. As expected, this protection is not reasonable, since typical DP-training guarantees the “membership” privacy of the trained final model, not (and far from) intermediate gradients. To measure on the pre-gradient level, one needs to adjust the DP’s granularity from sample-level (i.e., sensitivity of samples in the dataset) to feature-level (i.e., sensitivity of features in the sample). This level of DP noises will highly likely destroy the model utility but potentially provide a theoretical-based sense of the level of privacy for such protection.

Another line of work is to apply computational constraint-based $\mathcal{V}$ *usable information* theory [253]. $\mathcal{V}$ -information allows us to consider an attack family with bound computational power and limited knowledge. With the defined attack family, one can measure the capability of the complete attack family, which may include many attacks, by computing $\mathcal{V}$ -information similar to Shannon mutual information. This $\mathcal{V}$ -information then reflects the privacy leakage [177]. Although such a method provides a theoretical base for measuring privacy, how to define a broad and general attack family still deserves further exploration.

Different types of privacy in ML are fragmental and overlapped, as concluded based on both previous empirical attacks and theoretical analyses. Without a solid understanding of privacy, TEE protection itself is not plausible; protection of one type of privacy could be invalid when the privacy goal changes. Therefore, a rigorous definition of privacy for specific use cases is the cornerstone for TEE protection.

7.2 Partitioned ML Execution for Heterogeneous TEEs

For now and in the foreseeable future, most TEEs on modern devices will be heterogeneous, and many will keep small, considering the ubiquitousness of devices and the requirement of small TCBs. It is desired to avoid complex monolithic ML frameworks; so, we could evaluate and partition the solution with a focus on protecting the most critical ML computation or stages.

The partition can be done on different scales. On a low level, most ML’s computation forms with numeral calculations, e.g., multiply-accumulate operations, floating point operations, or matrix multiplication. Running a portion of these computations inside TEE is straightforward, but determining whether computation is not an easy task. First, this determination needs well-defined privacy as above-mentioned, e.g., which type of privacy to protect. Second, it requires the knowledge of how these computations have different leakage of the target privacy. Currently, we have a superficial understanding of layer-level privacy differences, i.e., the last layer contains the most membership privacy. However, there is still a need (i) to deepen the layer-level understanding of more types of privacy; (ii) to investigate efficient layer-wise partitioning approaches; (iii) to investigate any more fine-grained privacy understanding (i.e., matrix multiplication and feature maps), as the layer-level partition is still heavy for constrained TEEs. Furthermore, one can also execute some kinds of assurance and verification inside TEEs instead of hiding sensitive parts directly. Such a method has been used for ML integrity checking [183] but still deserves further investigation, i.e., combining with masking techniques to preserve privacy.

7.3 Dedicated TEE Designs for General ML

Efficient ML execution requires modern computer architectures with parallelization features, e.g., multi-threading on CPUs to load data and on GPUs (or TPU, etc.) to perform matrix multiplication in forward and backward passes. These features will need dedicated (co)designs of the

hardware and the system of accelerated processing units, which most of the current TEEs do not have. One solution is to equip TEEs with such parallelization features to increase the computational capability of ML. However, consequently, new features increase the TCB size. Synchronization bugs cause severe vulnerability of existing TEEs [168], and one can also expect a considerable number of bugs that potentially exist in GPU-TEEs. How to avoid such a dilemma really depends on how can we limit the trust boundary and reduce the TCB when applying parallel processing. One example is to consider one ML accelerated unit (GPU/TPU) to be physically isolated from the rest of the main board and the processing system in the same way as TPM, and to be accessed only through a secure bus from TEE-enabled CPUs. Although this constrains the trust boundary with assumptions that this dedicated GPU serves only for such private ML and no one can physically breach the GPU, there are more use cases applicable, such as MLaaS with proper remote attestation and verification. Recent work on designing secure IO paths and attestation support for interactions between various confidential VMs (e.g., TDX, SNP-SEV, CCA) and confidential GPUs shows great progress toward trustworthy systems dedicated to ML workloads [254, 255].

Interestingly, aside from acceleration via core processors, the arising technique called in-memory computing [256, 257] offers another way for speedy data processing, also for ML [256, 258, 259]. As an example, Mythic introduced a memory architecture for in-memory matrix multiplication. Compared to traditional computing architecture, it avoids a large amount of data transmitted across CPUs, SRAM, and DRAM, which could then speed up to 100× in some cases. This potentially leads to interesting interactions with the most recent TEE designs (SGXv2 and Arm CCA) that support large memory sizes.

7.4 TEE-aware ML

Due to the heterogeneity of devices' TEEs and ML workload, it is obvious that one solution cannot satisfy all needs. Therefore, designing the ML to be aware of TEEs and the capability of TEEs is critical. To achieve this, ML frameworks will determine the maximum workload to be executed with TEEs, which first requires the TEE's information such as memory size, processor speed, protected storage, and potentially other features such as multi-threading and secure communication channels. Then, ML computations from the most sensitive to less are deployed into TEEs. Such a determination involves many variables that influence factors from system cost to ML performance; thus, one can potentially form a multi-objective optimization problem similar to Neural Architecture Search for resource-constrained environments [260–262]. Furthermore, one may need techniques such as sparse approach [263, 264] of knowledge distillation [265, 266] to handle the heterogeneity of TEEs and differences in model architectures, especially when updating models in FL.

7.5 Protecting the Full ML Pipeline

Protecting the full pipeline is intractable without the participation of multiple TEEs (e.g., References [34, 267]). Multiple TEEs are able to provide multiple trusted areas covering more devices and consequently more pieces of the pipeline. To achieve such “multi-party computation” based on TEEs, one needs to enable a verification mechanism for multiple participants so TEEs from different chip manufacturers can be remotely attested by two or more challengers. For example, potentially one third-party proxy attestation service can help to verify TEEs from both Arm and Intel [124] so the TEE system could cover multiple locations of the ML pipeline. Also, we anticipate more usage of virtualization-based enclaves (such as Microsoft VBS enclaves [282]) on edge devices that do not have proper support for hardware enclaves. This approach introduces interesting opportunities for achieving more flexible and pluggable security models but also introduces new security challenges that are not fully explored yet.

In addition to the TEE workflow, determining the most sensitive parts of ML for protection is also not trivial. On the smallest scale, TEE provides a more trustworthy area in the ML pipeline, such as an additional trust base for random number producer or DP noise addition). Besides, investigations on the privacy or integrity of the pipeline's parts other than training/inference protection, such as data preparation, are vastly missing [268]. Such protection on one specific part of the pipeline will involve again threat (privacy/integrity) definition, sophisticated TEE-based protection design, and performance measurement similar to the training stage protection. After such works on more stages of the pipeline, full ML pipeline protection will be formed concretely and be used to a greater extent and larger scale.

7.6 Risk of Side-channel Attacks

Additionally, although it is out of scope in typical threat models of Confidential Computing, the risk of side-channel attacks would be increased due to the increasingly complex process when incorporating the ML process, which should be better analyzed and handled. First, conducting side-channel attacks to disclose the structure of DNNs has been shown to be possible in various ways. Privado [156] and Hua's work [269] demonstrated that by observing memory access patterns a CPU-TEE accesses, the adversary can infer the DNN's network structure. GANRED [240] and Cache Telepathy [270] used only cache timing side-channel information and still were able to extract a targeting modern DNNs from a CPU-TEE. Similarly, **electromagnetic (EM)** emanations and power information also can be leveraged to disclose DNN information [242, 271].

Such vulnerability becomes severe with application-specific processors like CNN accelerators. For instance, Hermes [264] observed the data stream on the PCIe bus between GPUs and CPUs, and Leaky DNN [272] monitored the resource usage of the GPU's kernel; both can steal DNN models' hyper-parameters, even identical weight information. The same side-channel issue also exists in microcontrollers that have been massively used in pervasive applications such as wearables and surveillance cameras shown by Reference [242]'s attack. These potential and new rising side-channel attacks should be delicately examined when implementing a Confidential Computing system for ML. Still, there are countermeasures that transform applications (e.g., ML frameworks) to be free of input/output-dependent access patterns by introducing obliviousness into the input/output. Though recent work shows progress in various mitigation techniques [273], we are not even close to bridging this gap; hence, we anticipate further developments tackling side-channel risks to emerge in the future.

7.7 Comparative Analysis and Broader Impacts

Liu et al. [274] give a solid survey on privacy-related works with ML, including both ML for privacy and private ML. However, the survey considers only encryption-based protection, i.e., homomorphic encryption, multi-party computation, differential privacy, and so on, and does not include any hardware-based protection mechanism such as TEEs. TEEs are becoming one of the prominent protection mechanisms, along with the act of GPU manufacturer NVIDIA, and are much more efficient than these encryption-based protections, which is our main target in this survey. Note that in the previous work [274], attacks are grouped similarly to our categories but mostly following the name of the original work. The Model Extraction Attack refers to stealing the model, corresponding to model privacy; the Model Inversion Attack and Shadow Model Attack refer to inferring certain attributes or statistical properties of the training dataset, corresponding to AIA; the Model Memorization Attack refers to recovering the exact input data, corresponding to DRA. However, the original names are more or less inaccurate. For instance, the phrases "Model Inversion Attack" and "Model Memorization Attack" both implicitly express that the attack leverages the models' information to invert/reverse the information of others like training data, but are not

distinguished on whether the information is exact data or features/attributes of the data. Our work aligns with more recent surveys on privacy attacks in ML [275–277] and NIST’s publication on the taxonomy and terminology of ML attacks [95], and we appeal for these more explicit taxonomies in future research.

Both previous surveys cited in References [39] and [278] dedicatedly summarize Confidential Computing-assisted ML with detailed descriptions of both TEE and ML scenarios. However, they have not examined the complete ML pipeline; therefore, important components may be missing, such as (i) the software stack that connects the TEE hardware stack to ML computation and (ii) the input data that is always outside of TEE, which, in contrast, has been comprehensively analyzed in our survey. Our work not only performs a more comprehensive summarization of previous work but also gives a “bird’s-eye view” of the interdisciplinary to identify the necessary components to close the gap between Confidential Computing and ML. Consequently, our survey presents more insights into underlying challenges, limitations of existing systems, and potential solutions, which realize this work as a Systematization of Knowledge.

From a broader perspective, ML with Confidential Computing could potentially provide solutions for many regulations, such as GDPR [279], CCPA [280], and EU AI Act [281]. As stated in the EU AI Act, existing EU law on the protection of privacy and confidentiality still applies to the use of information for AI-based technologies. TEEs establish enclaves with restricted access for processing personal data, which consequently provides a level of privacy guarantees and confidentiality assurance, i.e., “the right to restrict processing” in GDPR. Interestingly, with the above-mentioned privacy measures (in Section 7.1), we gain a better understanding, both theoretically and empirically, of the level of information contained in the output of the TEE. This also brings forth GDPR’s “the right to explanation” of the AI algorithm’s outputs to a certain extent. We believe with advanced techniques being developed in the future, Confidential Computing will be better integrated with ML and move forward in real-world applications while satisfying regulations.

8 Conclusion

Protecting ML against privacy leakages and integrity breaches using Confidential Computing techniques is an exciting and challenging new era. Although many studies have been dedicated to running the training and inference processes inside TEEs, they still face the limitation of available trust resources. Since ML requires much more trusted resources, the current protection levels only provide the confidentiality and integrity of that specific stage, training/inference, in the complete ML pipeline. Current defenses against privacy leakage are usually measured through red-teaming and attack-specific metrics, which make the final results less reliable and generalized. Furthermore, large attack surfaces exist at stages, especially at the upstream stage like data preparation, which can cause huge negative impacts on the ML pipeline. Rethinking how to tackle such a dilemma, i.e., the conflict between the large scale of the complete ML pipeline and the need for small TCB, is of utmost importance when conducting ML with Confidential Computing.

Confidential Computing achieves a hardware-based root-of-trust establishing a more trustworthy execution environment for ML activities, but we should rethink whether “hiding” the training/inference process inside such enclaves is the optimal solution. Future researchers/developers need to better understand the underlying privacy issues in the ML pipeline so later protections focus on vital parts. The full ML pipeline needs to be evaluated to avoid fruitless labor on training/inference protection only. The current trends indicated that both TEE evolution and ML advances (i.e., partitioning ML execution for TEEs and designing dedicated TEEs for ML) will lead to more *TEE-aware* future architectures. These potential solutions can help achieve much stronger privacy and security guarantees without compromising ML computational performance and introducing TEE’s system overhead.

Suggestions to achieve ML with Confidential Computing for the hardware/system/framework design are:

- Compared to security or integrity, privacy definition is still ambiguous. For a targeted privacy guarantee goal, one needs to apply privacy definitions with strong theoretical foundations, such as differential privacy or $\mathcal{V}$ -usable information.
- Protecting the upstream of the ML pipeline such as the data preparation is of utmost importance, because its lack has intractable impacts on the downstream ML process. It can potentially be realized by integrating TEE-based verification into the data signature. Multiple TEEs/Conclaves can further benefit the full ML pipeline protection.
- Developing dedicated TEEs for general ML requires rethinking and redesigning the hardware acceleration features and corresponding system stack. New paradigms such as in-memory computing that involves a smaller group of hardware and software modules may ease the condition due to the reduction of the TCB size and the attack surface.
- Designing the ML framework to be TEE-aware and partitionable for heterogeneous TEEs requires (i) meticulously researching the privacy/integrity weakness of different ML granularity (from layer/feature map to numeral calculations) and (ii) managing the TEE system to protect the most sensitive ML components efficiently (e.g., with masking) with a high priority.

References

- [1] Roberto Boselli, Mirko Cesarini, Fabio Mercorio, and Mario Mezzanzanica. 2018. Classifying online job advertisements through machine learning. *Fut. Gen. Comput. Syst.* 86 (2018), 319–328.
- [2] Maxim Naumov, Dheevatsa Mudigere, Hao-Jun Michael Shi, Jianyu Huang, Narayanan Sundaraman, Jongsoo Park, Xiaodong Wang, Udit Gupta, Carole-Jean Wu, Alisson G. Azzolini, Dmytro Dzhulgakov, Andrey Mallevich, Ilia Cherniavskii, Yinghai Lu, Raghuraman Krishnamoorthi, Ansha Yu, Volodymyr Kondratenko, Stephanie Pereira, Xianjie Chen, Wenlin Chen, Vijay Rao, Bill Jia, Liang Xiong, and Misha Smelyanskiy. 2019. Deep learning recommendation model for personalization and recommendation systems. *arXiv preprint arXiv:1906.00091* (2019).
- [3] Martin Leo, Suneel Sharma, and Koilakuntla Maddulety. 2019. Machine learning in banking risk management: A literature review. *Risks* 7, 1 (2019), 29.
- [4] Muhammad Aurangzeb Ahmad, Carly Eckert, and Ankur Teredesai. 2018. Interpretable machine learning in healthcare. In *ACM International Conference on Bioinformatics, Computational Biology, and Health Informatics*. 559–560.
- [5] K. Shailaja, B. Seetharamulu, and M. A. Jabbar. 2018. Machine learning in healthcare: A review. In *2nd International Conference on Electronics, Communication and Aerospace Technology (ICECA'18)*. IEEE, 910–914.
- [6] Michael Meyer and Georg Kusch. 2019. Deep learning based 3D object detection for automotive radar and camera. In *16th European Radar Conference (EuRAD'19)*. IEEE, 133–136.
- [7] Ratheesh Ravindran, Michael J. Santora, and Mohsin M. Jamali. 2020. Multi-object detection and tracking, based on DNN, for autonomous vehicles: A review. *IEEE Sensors J.* 21, 5 (2020), 5668–5677.
- [8] Tribhuvanesh Orekondy, Bernt Schiele, and Mario Fritz. 2019. Knockoff nets: Stealing functionality of black-box models. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 4954–4963.
- [9] Florian Tramèr, Fan Zhang, Ari Juels, Michael K. Reiter, and Thomas Ristenpart. 2016. Stealing machine learning models via prediction APIs. In *25th USENIX Security Symposium (USENIX Security'16)*. 601–618.
- [10] Yun Shen, Xinlei He, Yufei Han, and Yang Zhang. 2022. Model stealing attacks against inductive graph neural networks. In *IEEE Symposium on Security and Privacy (SP'22)*. IEEE, 1175–1192.
- [11] Adnan Siraj Rakin, Md Hafizul Islam Chowdhury, Fan Yao, and Deliang Fan. 2022. DeepSteal: Advanced model extractions leveraging efficient weight stealing in memories. In *IEEE Symposium on Security and Privacy (SP'22)*. IEEE, 1157–1174.
- [12] Matt Fredrikson, Somesh Jha, and Thomas Ristenpart. 2015. Model inversion attacks that exploit confidence information and basic countermeasures. In *22nd ACM SIGSAC Conference on Computer and Communications Security*. 1322–1333.
- [13] Zecheng He, Tianwei Zhang, and Ruby B. Lee. 2019. Model inversion attacks against collaborative inference. In *35th Annual Computer Security Applications Conference*. 148–162.
- [14] Minghong Fang, Xiaoyu Cao, Jinyuan Jia, and Neil Gong. 2020. Local model poisoning attacks to byzantine-robust federated learning. In *29th USENIX Security Symposium (USENIX Security'20)*. 1605–1622.

- [15] Xinyun Chen, Chang Liu, Bo Li, Kimberly Lu, and Dawn Song. 2017. Targeted backdoor attacks on deep learning systems using data poisoning. *arXiv preprint arXiv:1712.05526* (2017).
- [16] Virat Shejwalkar, Amir Houmansadr, Peter Kairouz, and Daniel Ramage. 2022. Back to the drawing board: A critical evaluation of poisoning attacks on production federated learning. In *IEEE Symposium on Security and Privacy (SP'22)*. IEEE, 1354–1371.
- [17] Ligeng Zhu, Zhijian Liu, and Song Han. 2019. Deep leakage from gradients. *Advan. Neural Inf. Process. Syst.* 32 (2019), 14774–14784.
- [18] Hongxu Yin, Arun Mallya, Arash Vahdat, Jose M. Alvarez, Jan Kautz, and Pavlo Molchanov. 2021. See through gradients: Image batch recovery via GradInversion. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 16337–16346.
- [19] Reza Shokri, Marco Stronati, Congzheng Song, and Vitaly Shmatikov. 2017. Membership inference attacks against machine learning models. In *IEEE Symposium on Security and Privacy (SP'17)*. IEEE, 3–18.
- [20] Luca Melis, Congzheng Song, Emiliano De Cristofaro, and Vitaly Shmatikov. 2019. Exploiting unintended feature leakage in collaborative learning. In *IEEE Symposium on Security and Privacy (SP'19)*. IEEE, 691–706.
- [21] Nicholas Carlini, Steve Chien, Milad Nasr, Shuang Song, Andreas Terzis, and Florian Tramer. 2022. Membership inference attacks from first principles. In *IEEE Symposium on Security and Privacy (SP'22)*. IEEE, 1897–1914.
- [22] Ian J. Goodfellow, Jonathon Shlens, and Christian Szegedy. 2014. Explaining and harnessing adversarial examples. *arXiv preprint arXiv:1412.6572* (2014).
- [23] Thore Graepel, Kristin Lauter, and Michael Naehrig. 2012. ML confidential: Machine learning on encrypted data. In *International Conference on Information Security and Cryptology*. Springer, 1–21.
- [24] David W. Archer, Borja de Balle Pigem, Dan Bogdanov, Mark Craddock, Adria Gascon, Ronald Jansen, Matjaž Jug, Kim Laine, Robert McLellan, Olga Ohrimenko, Mariana Raykova, Andrew Trask, and Simon Wardley. 2019. UN handbook on privacy-preserving computation techniques. Available: <https://unstats.un.org/bigdata/task-teams/privacy/UN%20Handbook%20for%20Privacy-Preserving%20Techniques.pdf>
- [25] Christopher A. Choquette-Choo, Natalie Dullerud, Adam Dziedzić, Yunxiang Zhang, Somesh Jha, Nicolas Papernot, and Xiao Wang. 2021. CaPC learning: Confidential and private collaborative learning. *arXiv preprint arXiv:2102.05188* (2021).
- [26] The Linux Foundation Project. 2023. Confidential Computing Consortium. Retrieved 08-June-2024 from <https://confidentialcomputing.io/>
- [27] Confidential Computing Consortium. 2023. A Technical Analysis of Confidential Computing v1.3. Retrieved 08-June-2024 from https://confidentialcomputing.io/wp-content/uploads/sites/10/2023/03/CCC-A-Technical-Analysis-of-Confidential-Computing-v1.3_unlocked.pdf
- [28] Confidential Computing Consortium. Confidential Computing: Hardware-based Trusted Execution for Applications and Data.
- [29] Olga Ohrimenko, Felix Schuster, Cédric Fournet, Aastha Mehta, Sebastian Nowozin, Kapil Vaswani, and Manuel Costa. 2016. Oblivious multi-party machine learning on trusted processors. In *25th USENIX Security Symposium (USENIX Security'16)*. 619–636.
- [30] Florian Tramer and Dan Boneh. 2018. Slalom: Fast, verifiable and private execution of neural networks in trusted hardware. In *International Conference on Learning Representations*.
- [31] Tyler Hunt, Congzheng Song, Reza Shokri, Vitaly Shmatikov, and Emmett Witchel. 2018. Chiron: Privacy-preserving machine learning as a service. *arXiv preprint arXiv:1803.05961* (2018).
- [32] Taegyeong Lee, Zhiqi Lin, Saumay Pushp, Caihua Li, Yunxin Liu, Youngki Lee, Fengyuan Xu, Chenren Xu, Lintao Zhang, and Junehwa Song. 2019. Occlumency: Privacy-preserving remote deep-learning inference using SGX. In *25th Annual International Conference on Mobile Computing and Networking*. 1–17.
- [33] Fan Mo and Hamed Haddadi. 2019. Efficient and private federated learning using TEE. In *European Conference on Computer Systems (EuroSys'19)*.
- [34] Fan Mo, Hamed Haddadi, Kleomenis Katevas, Eduard Marin, Diego Perino, and Nicolas Kourtellis. 2021. PPFL: Privacy-preserving federated learning with trusted execution environments. *arXiv preprint arXiv:2104.14380* (2021).
- [35] Fan Mo. 2022. *Privacy-preserving Machine Learning System at the Edge*. Ph.D. Dissertation. Imperial College London.
- [36] Microsoft. 2024. Azure Confidential Computing. Retrieved 08-June-2024 from <https://azure.microsoft.com/en-gb/solutions/confidential-compute/#overview>
- [37] Amazon Web Services. 2024. AWS Nitro Enclaves: Create Additional Isolation to Further Protect Highly Sensitive Data within EC2 Instances. Retrieved 08-June-2024 from <https://aws.amazon.com/ec2/nitro/nitro-enclaves/>
- [38] Google Cloud. 2024. Google Cloud Confidential Computing. Retrieved 08-June-2024 from <https://cloud.google.com/confidential-computing>
- [39] Kha Dinh Duy, Taehyun Noh, Siwon Huh, and Hojoon Lee. 2021. Confidential machine learning computation in untrusted environments: A systems security perspective. *IEEE Access* 9 (2021), 168656–168677.

- [40] Michael I. Jordan and Tom M. Mitchell. 2015. Machine learning: Trends, perspectives, and prospects. *Science* 349, 6245 (2015), 255–260.
- [41] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. 2015. Deep learning. *Nature* 521, 7553 (2015), 436–444.
- [42] Saleema Amershi, Andrew Begel, Christian Bird, Robert DeLine, Harald Gall, Ece Kamar, Nachiappan Nagappan, Bemira Nushi, and Thomas Zimmermann. 2019. Software engineering for machine learning: A case study. In *IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP'19)*. IEEE, 291–300.
- [43] IBM. 2024. AI Model Lifecycle Management: Overview. Retrieved 08-June-2024 from <https://www.ibm.com/cloud/blog/ai-model-lifecycle-managementoverview>
- [44] ETSI. 2020. Securing Artificial Intelligence (SAI) Problem Statement. Retrieved 08-June-2024 from https://www.etsi.org/deliver/etsi_gr/SAI/001_099/004/01.01.01_60/gr_SAI004v010101p.pdf
- [45] Xiaojin Zhu and Andrew B. Goldberg. 2009. Introduction to semi-supervised learning. *Synth. Lect. Artif. Intell. Mach. Learn.* 3, 1 (2009), 1–130.
- [46] David Berthelot, Nicholas Carlini, Ian Goodfellow, Nicolas Papernot, Avital Oliver, and Colin A. Raffel. 2019. Mix-Match: A holistic approach to semi-supervised learning. *Advan. Neural Inf. Process. Syst.* 32 (2019).
- [47] Quoc V. Le. 2013. Building high-level features using large scale unsupervised learning. In *IEEE International Conference on Acoustics, Speech and Signal Processing*. IEEE, 8595–8598.
- [48] Jesper E. Van Engelen and Holger H. Hoos. 2020. A survey on semi-supervised learning. *Mach. Learn.* 109, 2 (2020), 373–440.
- [49] Michael Tschannen, Olivier Bachem, and Mario Lucic. 2018. Recent advances in autoencoder-based representation learning. *arXiv preprint arXiv:1812.05069* (2018).
- [50] Arash Vahdat and Jan Kautz. 2020. NVAE: A deep hierarchical variational autoencoder. *Advan. Neural Inf. Process. Syst.* 33 (2020), 19667–19679.
- [51] Richard S. Sutton and Andrew G. Barto. 2018. *Reinforcement Learning: An Introduction*. MIT Press.
- [52] Connor Shorten and Taghi M. Khoshgoftaar. 2019. A survey on image data augmentation for deep learning. *J. Big Data* 6, 1 (2019), 1–48.
- [53] Qizhe Xie, Zihang Dai, Eduard Hovy, Thang Luong, and Quoc Le. 2020. Unsupervised data augmentation for consistency training. *Advan. Neural Inf. Process. Syst.* 33 (2020).
- [54] Richard Evans and Edward Grefenstette. 2018. Learning explanatory rules from noisy data. *J. Artif. Intell. Res.* 61 (2018), 1–64.
- [55] Angelos Katharopoulos and François Fleuret. 2018. Not all samples are created equal: Deep learning with importance sampling. In *International Conference on Machine Learning*. PMLR, 2525–2534.
- [56] Sören Mindermann, Muhammed Razzak, Mrinank Sharma, Jan M. Brauner, Winnie Xu, Andreas Kirsch, Aidan Gomez, Benedikt Hölting, Sebastian Farquhar, and Yarin Gal. 2022. Prioritized training on points that are learnable, worth learning, and not yet learnt. In *International Conference on Machine Learning*. PMLR, 15630–15649.
- [57] Thomas Elsken, Jan Hendrik Metzen, and Frank Hutter. 2019. Neural architecture search: A survey. *J. Mach. Learn. Res.* 20, 1 (2019), 1997–2017.
- [58] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. 1998. Gradient-based learning applied to document recognition. *Proc. IEEE* 86, 11 (1998), 2278–2324.
- [59] Léon Bottou. 2010. Large-scale machine learning with stochastic gradient descent. In *19th International Conference on Computational Statistics (COMPSTAT'10)*. Springer, 177–186.
- [60] Ron Kohavi. 1995. A study of cross-validation and bootstrap for accuracy estimation and model selection. In *International Joint Conference on Artificial Intelligence*. 1137–1145.
- [61] Sylvain Arlot and Alain Celisse. 2010. A survey of cross-validation procedures for model selection. *Stat. Surv.* 4 (2010), 40–79.
- [62] Sinno Jialin Pan and Qiang Yang. 2009. A survey on transfer learning. *IEEE Trans. Knowl. Data Eng.* 22, 10 (2009), 1345–1359.
- [63] Pedro Marcelino. 2018. Transfer learning from pre-trained models. *Towards Data Sci.* (2018).
- [64] Jason Yosinski, Jeff Clune, Yoshua Bengio, and Hod Lipson. 2014. How transferable are features in deep neural networks? *arXiv preprint arXiv:1411.1792* (2014).
- [65] Matthew D. Zeiler and Rob Fergus. 2014. Visualizing and understanding convolutional networks. In *European Conference on Computer Vision*. Springer, 818–833.
- [66] Hang Qi, Matthew Brown, and David G. Lowe. 2018. Low-shot learning with imprinted weights. In *IEEE Conference on Computer Vision and Pattern Recognition*. 5822–5830.
- [67] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. 2017. Communication-efficient learning of deep networks from decentralized data. In *Artificial Intelligence and Statistics*. PMLR, 1273–1282.
- [68] Tian Li, Anit Kumar Sahu, Ameet Talwalkar, and Virginia Smith. 2020. Federated learning: Challenges, methods, and future directions. *IEEE Sig. Process. Mag.* 37, 3 (2020), 50–60.

- [69] Peter Kairouz, H. Brendan McMahan, Brendan Avent, Aurélien Bellet, Mehdi Bennis, Arjun Nitin Bhagoji, Kallista Bonawit, Zachary Charles, Graham Cormode, Rachel Cummings, Rafael G. L. D'Oliveira, Hubert Eichner, Salim El Rouayheb, David Evans, Josh Gardner, Zachary Garrett, Adrià Gascón, Badih Ghazi, Phillip B. Gibbons, Marco Gruteser, Zaid Harchaoui, Chaoyang He, Lie He, Zhouyuan Huo, Ben Hutchinson, Justin Hsu, Martin Jaggi, Tara Javidi, Gauri Joshi, Mikhail Khodak, Jakub Konečný, Aleksandra Korolova, Farinaz Koushanfar, Sanmi Koyejo, Tan-crède Lepoint, Yang Liu, Prateek Mittal, Mehryar Mohri, Richard Nock, Ayfer Özgür, Rasmus Pagh, Hang Qi, Daniel Ramage, Ramesh Raskar, Mariana Raykova, Dawn Song, Weikang Song, Sebastian U. Stich, Ziteng Sun, Ananda Theertha Suresh, Florian Tramèr, Praneeth Vepakomma, Jianyu Wang, Li Xiong, Zheng Xu, Qiang Yang, Felix X. Yu, Han Yu, and Sen Zhao. 2019. Advances and open problems in federated learning. *arXiv preprint arXiv:1912.04977* (2019).
- [70] Martin Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghe-mawat, Geoffrey Irving, Michael Isard, Manjunath Kudlur, Josh Levenberg, Rajat Monga, Sherry Moore, Derek G. Murray, Benoit Steiner, Paul Tucker, Vijay Vasudevan, Pete Warden, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. 2016. TensorFlow: A system for large-scale machine learning. In *12th USENIX Symposium on Operating Sys-tems Design and Implementation (OSDI'16)*. USENIX Association, Savannah, GA, 265–283. <https://www.usenix.org/conference/osdi16/technical-sessions/presentation/abadi>
- [71] Yuanshun Yao, ZhuJun Xiao, Bolun Wang, Bimal Viswanath, Haitao Zheng, and Ben Y. Zhao. 2017. Complexity vs. performance: Empirical analysis of machine learning as a service. In *Internet Measurement Conference*. 384–397.
- [72] Mu Li, David G. Andersen, Jun Woo Park, Alexander J. Smola, Amr Ahmed, Vanja Josifovski, James Long, Eugene J. Shekita, and Bor-Yiing Su. 2014. Scaling distributed machine learning with the parameter server. In *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI'14)*. 583–598.
- [73] Jinhyun So, Basak Guler, and Salman Avestimehr. 2020. A scalable approach for privacy-preserving collaborative machine learning. *Advan. Neural Inf. Process. Syst.* 33 (2020), 8054–8066.
- [74] Joost Verbraken, Matthijs Wolting, Jonathan Katzy, Jeroen Kloppenburg, Tim Verbelen, and Jan S. Rellermeyer. 2020. A survey on distributed machine learning. *ACM Computing Surv.* 53, 2 (2020), 1–33.
- [75] Tian Li, Anit Kumar Sahu, Manzil Zaheer, Maziar Sanjabi, Ameet Talwalkar, and Virginia Smith. 2020. Federated optimization in heterogeneous networks. In *Conference on Machine Learning and Systems (MLSys'20)*.
- [76] Sashank Reddi, Zachary Charles, Manzil Zaheer, Zachary Garrett, Keith Rush, Jakub Konečný, Sanjiv Kumar, and H. Brendan McMahan. 2020. Adaptive federated optimization. *arXiv preprint arXiv:2003.00295* (2020).
- [77] Hongyi Wang, Mikhail Yurochkin, Yuekai Sun, Dimitris Papailiopoulos, and Yasaman Khazaeni. 2020. Federated learning with matched averaging. *arXiv preprint arXiv:2002.06440* (2020).
- [78] Jonas Geiping, Hartmut Bauermeister, Hannah Dröge, and Michael Moeller. 2020. Inverting gradients—How easy is it to break privacy in federated learning? *arXiv preprint arXiv:2003.14053* (2020).
- [79] Mauro Ribeiro, Katarina Grolinger, and Miriam A. M. Capretz. 2015. MLaaS: Machine learning as a service. In *IEEE 14th International Conference on Machine Learning and Applications (ICMLA'15)*. IEEE, 896–902.
- [80] Ehsan Hesamifard, Hassan Takabi, Mehdi Ghasemi, and Rebecca N. Wright. 2018. Privacy-preserving machine learn-ing as a service. *Proc. Privac. Enhanc. Technol.* 3 (2018), 123–142.
- [81] Milad Nasr, Reza Shokri, and Amir Houmansadr. 2019. Comprehensive privacy analysis of deep learning: Passive and active white-box inference attacks against centralized and federated learning. In *IEEE Symposium on Security and Privacy (SP'19)*. IEEE, 739–753.
- [82] Briland Hitaj, Giuseppe Ateniese, and Fernando Perez-Cruz. 2017. Deep models under the GAN: Information leakage from collaborative deep learning. In *ACM SIGSAC Conference on Computer and Communications Security*. 603–618.
- [83] Jinyuan Jia and Neil Zhenqiang Gong. 2018. AttriGuard: A practical defense against attribute inference attacks via adversarial machine learning. In *27th USENIX Security Symposium (USENIX Security'18)*. 513–529.
- [84] Honggang Yu, Kaichen Yang, Teng Zhang, Yun-Yun Tsai, Tsung-Yi Ho, and Yier Jin. 2020. CloudLeak: Large-scale deep learning models stealing through adversarial examples. In *Network and Distributed System Security Symposium (NDSS'20)*.
- [85] Seyed-Mohsen Moosavi-Dezfooli, Alhussein Fawzi, Omar Fawzi, and Pascal Frossard. 2017. Universal adversarial perturbations. In *IEEE Conference on Computer Vision and Pattern Recognition*. 1765–1773.
- [86] Zahra Tarkhani, Lorena Qendro, Malachy O'Connor Brown, Oscar Hill, Cecilia Mascolo, and Anil Madhavapeddy. 2022. Enhancing the security & privacy of wearable brain-computer interfaces. *arXiv preprint arXiv:2201.07711* (2022).
- [87] Jose Rodrigo Sanchez Vicarte, Benjamin Schreiber, Riccardo Paccagnella, and Christopher W. Fletcher. 2020. Game of threads: Enabling asynchronous poisoning attacks. In *25th International Conference on Architectural Support for Programming Languages and Operating Systems*. 35–52.
- [88] Vale Tolpegin, Stacey Truex, Mehmet Emre Gursoy, and Ling Liu. 2020. Data poisoning attacks against federated learning systems. In *European Symposium on Research in Computer Security*. Springer, 480–501.

- [89] Yunfei Liu, Xingjun Ma, James Bailey, and Feng Lu. 2020. Reflection backdoor: A natural backdoor attack on deep neural networks. In *European Conference on Computer Vision*. Springer, 182–199.
- [90] Eugene Bagdasaryan, Andreas Veit, Yiqing Hua, Deborah Estrin, and Vitaly Shmatikov. 2020. How to backdoor federated learning. In *International Conference on Artificial Intelligence and Statistics*. PMLR, 2938–2948.
- [91] Jinyuan Jia, Yupei Liu, and Neil Zhenqiang Gong. 2022. BadEncoder: Backdoor attacks to pre-trained encoders in self-supervised learning. In *IEEE Symposium on Security and Privacy (SP'22)*. IEEE, 2043–2059.
- [92] Nicolas Papernot, Patrick McDaniel, Arunesh Sinha, and Michael P. Wellman. 2018. SoK: Security and privacy in machine learning. In *IEEE European Symposium on Security and Privacy (EuroS&P'18)*. IEEE, 399–414.
- [93] Alexandre Sablayrolles, Matthijs Douze, Cordelia Schmid, Yann Ollivier, and Hervé Jégou. 2019. White-box vs black-box: Bayes optimal strategies for membership inference. In *International Conference on Machine Learning*. PMLR, 5558–5567.
- [94] Samuel Yeom, Irene Giacomelli, Matt Fredrikson, and Somesh Jha. 2018. Privacy risk in machine learning: Analyzing the connection to overfitting. In *IEEE 31st Computer Security Foundations Symposium (CSF'18)*. IEEE, 268–282.
- [95] Apostol Vassilev, Alina Oprea, Alie Fordyce, and Hyrum Andersen. 2024. Adversarial machine learning: A taxonomy and terminology of attacks and mitigations. (2024).
- [96] Mark Russinovich, Manuel Costa, Cédric Fournet, David Chisnall, Antoine Delignat-Lavaud, Sylvan Clebsch, Kapil Vaswani, and Vikas Bhatia. 2021. Toward confidential cloud computing. *Commun. ACM* 64, 6 (2021), 54–61.
- [97] GlobalPlatform. 2015. *White Paper on the Trusted Execution Environment*. GlobalPlatform. Retrieved from https://globalplatform.org/wp-content/uploads/2018/04/GlobalPlatform_TEE_Whitepaper_2015.pdf
- [98] Le Guan, Peng Liu, Xinyu Xing, Xinyang Ge, Shengzhi Zhang, Meng Yu, and Trent Jaeger. 2017. TrustShadow: Secure execution of unmodified applications with ARM TrustZone. In *15th Annual International Conference on Mobile Systems, Applications, and Services*. ACM, 488–501.
- [99] Raad Bahmani, Ferdinand Brasser, Ghada Dessouky, Patrick Jauernig, Matthias Klimmek, Ahmad-Reza Sadeghi, and Emmanuel Stapf. 2021. CURE: A security architecture with CUsTomizable and Resilient Enclaves. In *30th USENIX Security Symposium (USENIX Security'21)*. 1073–1090.
- [100] David Kaplan. 2023. Hardware VM isolation in the cloud: Enabling confidential computing with AMD SEV-SNP technology. *Queue* 21, 4 (2023), 49–67.
- [101] Jaehyuk Lee, Jinsoo Jang, Yeongjin Jang, Nohyun Kwak, Yeseul Choi, Changho Choi, Taesoo Kim, Marcus Peinado, and Brent Byunghoon Kang. 2017. Hacking in darkness: Return-oriented programming against secure enclaves. In *26th USENIX Conference on Security Symposium*. 523–539.
- [102] Mustakimur Rahman Khandaker, Yueqiang Cheng, Zhi Wang, and Tao Wei. 2020. COIN attacks: On insecurity of enclave untrusted interfaces in SGX. In *25th International Conference on Architectural Support for Programming Languages and Operating Systems*. 971–985.
- [103] Jo Van Bulck, David Oswald, Eduard Marin, Abdulla Aldoseri, Flavio D. Garcia, and Frank Piessens. 2019. A tale of two worlds: Assessing the vulnerability of enclave shielding runtimes. In *ACM SIGSAC Conference on Computer and Communications Security (CCS'19)*. 1741–1758.
- [104] Zitai Chen, Georgios Vasilakis, Kit Murdock, Edward Dean, David Oswald, and Flavio D. Garcia. 2021. VoltPillager: Hardware-based fault injection attacks against Intel SGX Enclaves using the SVID voltage scaling interface. In *30th USENIX Security Symposium*.
- [105] Mengyuan Li, Luca Wilke, Jan Wichelmann, Thomas Eisenbarth, Radu Teodorescu, and Yinqian Zhang. 2022. A systematic look at ciphertext side channels on AMD SEV-SNP. In *IEEE Symposium on Security and Privacy (SP'22)*. IEEE Computer Society, 1541–1541.
- [106] Moritz Lipp, Andreas Kogler, David Oswald, Michael Schwarz, Catherine Easdon, Claudio Canella, and Daniel Gruss. 2021. PLATYPUS: Software-based power side-channel attacks on x86. In *IEEE Symposium on Security and Privacy (SP'21)*. IEEE.
- [107] Aravind Machiry, Eric Gustafson, Chad Spensky, Christopher Salls, Nick Stephens, Ruoyu Wang, Antonio Bianchi, Yung Ryn Choe, Christopher Kruegel, and Giovanni Vigna. 2017. BOOMERANG: Exploiting the semantic gap in trusted execution environments. In *Network and Distributed System Security Symposium (NDSS'17)*.
- [108] David Cerdeira, Nuno Santos, Pedro Fonseca, and Sandro Pinto. 2020. SoK: Understanding the prevailing security vulnerabilities in trustzone-assisted TEE systems. In *IEEE Symposium on Security and Privacy (SP'20)*. IEEE, 1416–1432.
- [109] Hongyan Xia, David Zhang, Wei Liu, Istvan Haller, Bruce Sherwin, and David Chisnall. 2022. A secret-free hypervisor: Rethinking isolation in the age of speculative vulnerabilities. In *IEEE Symposium on Security and Privacy (SP'22)*. IEEE Computer Society, 1544–1544.
- [110] Jatinder Singh, Jennifer Cobbe, Do Le Quoc, and Zahra Tarkhani. 2021. Enclaves in the clouds: Legal considerations and broader implications. *Commun. ACM* 64, 5 (2021), 42–51.

- [111] Microsoft. 2023. MS-IoT fTPM. Retrieved 08-June-2024 from <https://github.com/microsoft/ms-tpm-20-ref/blob/master/Samples/ARM32-FirmwareTPM/README.md>
- [112] Intel. 2024. Intel Software Guard Extensions SDK. Retrieved 08-June-2024 from <https://01.org/intel-software-guard-extensions>
- [113] Trusted Firmware. 2024. Open Portable Trusted Execution Environment. Retrieved 08-June-2024 from <https://www.op-tee.org/>
- [114] Hex Five Security. 2024. MultiZone Security for RISC-V. Retrieved 08-June-2024 from <https://hex-five.com/multizone-security-sdk/>
- [115] Keystone Enclave. 2021. Keystone: An Open Framework for Architecting Trusted Execution Environments. Retrieved 08-June-2024 from <https://keystoneenclave.org/>
- [116] Open Enclave. 2024. Open Enclave SDK. Retrieved 08-June-2024 from <https://openenclave.io/sdk/>
- [117] Asylo. 2018. Asylo: An Open and Flexible Framework for Enclave Applications. Retrieved 08-June-2024 from <https://asylo.dev/>
- [118] Apache Software Foundation. 2020. Apache Teaclave (incubating). Retrieved 08-June-2024 from <https://teaclave.apache.org/>
- [119] Gramine. 2021. Gramine - A Library OS for Unmodified Applications. Retrieved 08-June-2024 from <https://grapheneproject.io/>
- [120] Occlum. 2020. Occlum - A Library OS Empowering Everyone to Run Every Application in Secure Enclaves. Retrieved 08-June-2024 from <https://occlum.io/>
- [121] Scontain. 2024. SCONTAIN - A Secure Container Environment. Retrieved 08-June-2024 from <https://scontain.com/>
- [122] Enarx. 2024. Enarx: Confidential Computing with WebAssembly. Retrieved 08-June-2024 from <https://enarx.dev/>
- [123] Yuekai Jia, Shuang Liu, Wenhao Wang, Yu Chen, Zhengde Zhai, Shoumeng Yan, and Zhengyu He. 2022. HyperEnclave: An open and cross-platform trusted execution environment. In *USENIX Annual Technical Conference (USENIX ATC'22)*. 437–454.
- [124] Veracruz. 2021. Veracruz: Privacy-Preserving Collaborative Compute. Retrieved 08-June-2024 from <https://veracruz-project.com/>
- [125] Victor Costan and Srinivas Devadas. 2016. Intel SGX explained. *IACR Cryptol. ePrint Arch.* 2016, 86 (2016), 1–118.
- [126] Vivek Haldar, Deepak Chandra, and Michael Franz. 2004. Semantic remote attestation: A virtual machine directed approach to trusted computing. In *USENIX Virtual Machine Research and Technology Symposium*.
- [127] George Coker, Joshua Guttman, Peter Loscocco, Amy Herzog, Jonathan Millen, Brian O'Hanlon, John Ramsdell, Ariel Segall, Justin Sheehy, and Brian Sniffen. 2011. Principles of remote attestation. *Int. J. Inf. Secur.* 10, 2 (2011), 63–81.
- [128] Vinnie Scarlata, Simon Johnson, James Beaney, and Piotr Zmijewski. 2018. Supporting third party attestation for Intel SGX with Intel data center attestation primitives. *White Paper* (2018).
- [129] Simon Johnson, Vinnie Scarlata, Carlos Rozas, Ernie Brickell, and Frank McKeen. 2016. Intel software guard extensions: EPID provisioning and attestation services. *White Paper* 1, 1-10 (2016), 119.
- [130] Intel. 2020. Intel® Trust Domain Extensions (Intel® TDX). Retrieved from <https://software.intel.com/content/www/us/en/develop/articles/intel-trust-domain-extensions.html>
- [131] Bernard Ngabonziza, Daniel Martin, Anna Bailey, Haehyun Cho, and Sarah Martin. 2016. TrustZone explained: Architectural features and use cases. In *IEEE 2nd International Conference on Collaboration and Internet Computing (CIC'16)*. IEEE, 445–451.
- [132] Xupeng Li, Xuheng Li, Christoffer Dall, Ronghui Gu, Jason Nieh, Yousuf Sait, and Gareth Stockwell. 2022. Design and verification of the Arm confidential compute architecture. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI'22)*. 465–484.
- [133] Arm. 2021. Confidential Compute Architecture: Placing Confidential Compute in the Hands of Every Developer. <https://haspworkshop.org/2021/slides/HASP-2021-Session2-Arm-CCA.pdf>
- [134] Dominic P. Mulligan, Gustavo Petri, Nick Spinale, Gareth Stockwell, and Hugo J. M. Vincent. 2021. Confidential computing—A brave new world. In *International Symposium on Secure and Private Execution Environment Design (SEED'21)*. IEEE, 132–138.
- [135] Xiangyi Xu, Wenhao Wang, Yongzheng Wu, Zhennan Min, Zixuan Pang, and Yier Jin. 2023. virtCCA: Virtualized Arm confidential compute architecture with TrustZone. *arXiv preprint arXiv:2306.11011* (2023).
- [136] Ravi Sahita, Vedvyas Shanbhogue, Andrew Bresticker, Atul Khare, Atish Patra, Samuel Ortiz, Dylan Reid, and Rajnesh Kanwal. 2023. CoVE: Towards confidential computing on RISC-V platforms. In *20th ACM International Conference on Computing Frontiers*. 315–321.
- [137] NXP Semiconductors. 2024. Trusted Execution Environment: Getting Started with OP-TEE on i.MX Processors. Retrieved 08-June-2024 from <https://www.nxp.com/design/training/trusted-execution-environment-getting-started-with-op-tee-on-i-mx-processors:TIP-TRUSTEDEXECUTION-ENVIRONMENT-GETTING-STARTED>

- [138] Andrew Baumann, Marcus Peinado, and Galen Hunt. 2015. Shielding applications from an untrusted cloud with Haven. *ACM Trans. Comput. Syst.* 33, 3 (2015), 8.
- [139] Donald E. Porter, Silas Boyd-Wickizer, Jon Howell, Reuben Olinsky, and Galen C. Hunt. 2011. Rethinking the library OS from the top down. In *ACM SIGPLAN Notices*, Vol. 46. ACM, 291–304.
- [140] Mathias Brossard, Guilhem Bryant, Basma El Gaabouri, Xinxin Fan, Alexandre Ferreira, Edmund Grimley Evans, Christopher Haster, Evan Johnson, Derek Miller, Fan Mo, Dominic P. Mulligan, Nick Spinale, Eric van Hensbergen, Hugo J. M. Vincent, and Shale Xiong. 2023. Private delegated computations using strong isolation. *IEEE Transactions on Emerging Topics in Computing* 12, 1 (2023), 386–398.
- [141] Chia-Che Tsai, Donald E. Porter, and Mona Vij. 2017. Graphene-SGX: A practical library OS for unmodified applications on SGX. In *USENIX Annual Technical Conference (ATC'17)*. 8.
- [142] Sergei Arnaudov, Bohdan Trach, Franz Gregor, Thomas Knauth, Andre Martin, Christian Priebe, Joshua Lind, Divya Muthukumar, Dan O'Keeffe, Mark L. Stillwell, David Goltzsche, Dave Eysers, Rüdiger Kapitza, Peter Pietzuch, and Christof Fetzer. 2016. SCONe: Secure Linux containers with Intel SGX. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI'16)*. 689–703. <https://www.usenix.org/conference/osdi16/technical-sessions/presentation/arnautov>
- [143] Imperial College London. 2019. SGX-LKL Library OS for running Linux applications inside of Intel SGX enclaves. Retrieved from <https://github.com/llds/sgx-lkl>
- [144] Microsoft. 2024. Deep dive: Secure orchestration of confidential containers on AKS. Retrieved from <https://techcommunity.microsoft.com/t5/linux-and-open-source-blog/deep-dive-secure-orchestration-of-confidential-containers-on/ba-p/4137179>
- [145] Google Cloud. 2024. Confidential GKE Nodes: Encrypting Data in Use. Retrieved 08-June-2024 from <https://cloud.google.com/kubernetes-engine/docs/how-to/confidential-gke-nodes>
- [146] Intel Corporation. 2019. Intel(R) Software Guard Extensions for Linux OS. Retrieved from <https://github.com/intel/linux-sgx>
- [147] Microsoft Corporation. 2019. Open Enclave SDK. Retrieved from <https://github.com/openenclave/openenclave>
- [148] Google. 2018. Asylo - An open and flexible framework for enclave applications. Retrieved from <http://web.archive.org/web/20080207010024/http://www.808multimedia.com/winnt/kernel.htm>
- [149] Dayeol Lee, David Kohlbrenner, Shweta Shinde, Krste Asanovic, and Dawn Song. 2020. Keystone: An open framework for architecting trusted execution environments. In *15th European Conference on Computer Systems (EuroSys'20)*.
- [150] Chia-Che Tsai, Jeongseok Son, Bhushan Jain, John McAvey, Raluca Ada Popa, and Donald E. Porter. 2020. Civet: An efficient Java partitioning framework for hardware enclaves. In *29th USENIX Security Symposium (USENIX Security'20)*.
- [151] Nuno Santos, Himanshu Raj, Stefan Saroiu, and Alec Wolman. 2014. Using ARM TrustZone to build a trusted language runtime for mobile applications. *ACM SIGARCH Comput. Archit. News* 42, 1 (2014), 67–80.
- [152] Joshua Lind, Christian Priebe, Divya Muthukumar, Dan O'Keeffe, Pierre-Louis Aublin, Florian Kelbert, Tobias Reiher, David Goltzsche, David Eysers, Rüdiger Kapitza, Christof Fetzer, and Peter Pietzuch. 2017. Glamdring: Automatic application partitioning for intel SGX. In *USENIX Annual Technical Conference (USENIX ATC'17)*. USENIX Association, Santa Clara, CA, 285–298. <https://www.usenix.org/conference/atc17/technicalsessions/presentation/lind>
- [153] Qualcomm. 2019. *Guard Your Data with the Qualcomm Snapdragon Mobile Platform*. Retrieved from <https://www.qualcomm.com/media/documents/files/guard-your-data-with-the-qualcomm-snapdragon-mobile-platform.pdf>
- [154] Trustonic. Trustonic: World leading embedded cybersecurity technology Trustonic. Retrieved from <https://www.trustonic.com/>
- [155] Marcel Busch, Johannes Westphal, and Tilo Mueller. 2020. Unearthing the TrustedCore: A critical review on Huawei's trusted execution environment. In *14th USENIX Workshop on Offensive Technologies (WOOT'20)*.
- [156] Karan Grover, Shruti Tople, Shweta Shinde, Ranjita Bhagwan, and Ramachandran Ramjee. 2018. Privado: Practical and secure DNN inference with enclaves. *arXiv preprint arXiv:1810.00602* (2018).
- [157] Abhinav Kumar, Reza Tourani, Mona Vij, and Srikathyayani Srikanteswara. 2022. SCLERA: A framework for privacy-preserving MLaaS at the pervasive edge. In *IEEE International Conference on Pervasive Computing and Communications Workshops and other Affiliated Events (PerCom Workshops'22)*. IEEE, 175–180.
- [158] Nick Hynes, Raymond Cheng, and Dawn Song. 2018. Efficient deep learning on multi-source private data. *arXiv preprint arXiv:1807.06689* (2018).
- [159] Sebastian P. Bayerl, Tommaso Frassetto, Patrick Jauernig, Korbinian Riedhammer, Ahmad-Reza Sadeghi, Thomas Schneider, Emmanuel Stapf, and Christian Weinert. 2020. Offline model guard: Secure and private ML on mobile devices. In *Design, Automation & Test in Europe Conference & Exhibition (DATE'20)*. IEEE, 460–465.
- [160] Fan Mo, Ali Shahin Shamsabadi, Kleomenis Katevas, Soteris Demetriou, Ilias Leontiadis, Andrea Cavallaro, and Hamed Haddadi. 2020. DarkneTZ: Towards model privacy at the edge using trusted execution environments. In *18th International Conference on Mobile Systems, Applications, and Services*. 161–174.

- [161] Frank McKeen, Ilya Alexandrovich, Ittai Anati, Dror Caspi, Simon Johnson, Rebekah Leslie-Hurd, and Carlos Rozas. 2016. Intel® software guard extensions (Intel® SGX) support for dynamic memory management inside an enclave. In *Hardware and Architectural Support for Security and Privacy Conference*. 1–9.
- [162] Intel2. SGX Intel® Xeon® Scalable Processors. Retrieved 08-June-2024 from <https://www.intel.com/content/www/us/en/architecture-and-technology/software-guard-extensions-processors.html>
- [163] Lenovo. 2023. Enabling Intel SGX on Lenovo ThinkSystem servers. Retrieved from <https://lenovopress.lenovo.com/lp1471.pdf>
- [164] Intel Corporation. 2024. Build Secure, Scalable, and Accelerated Machine Learning Pipelines — intel.com. Retrieved 08-June-2024 from <https://www.intel.com/content/www/us/en/developer/articles/troubleshooting/build-secure-scalable-accelerate-ml-pipelines.html>
- [165] Intel. 2024. Demetics Protects AI-Based Medical Innovation with Intel SGX — intel.com. Retrieved 08-June-2024 from <https://www.intel.com/content/www/us/en/newsroom/news/demetics-protects-ai-based-medical-innovation-sgx.html>
- [166] Intel. 2024. Alibaba Builds End-to-End PPML Solution — intel.com. Retrieved 08-June-2024 from <https://www.intel.com/content/www/us/en/customerspotlight/stories/alibaba-cloud-ppml-customer-story.html>
- [167] Roland Kunkel, Do Le Quoc, Franz Gregor, Sergei Arnautov, Pramod Bhatotia, and Christof Fetzer. 2019. TensorSCONE: A secure tensorflow framework using Intel SGX. *arXiv preprint arXiv:1902.04413* (2019).
- [168] Nico Weichbrodt, Anil Kurmus, Peter Pietzuch, and Rüdiger Kapitza. 2016. AsyncShock: Exploiting synchronisation bugs in Intel SGX enclaves. In *European Symposium on Research in Computer Security*. Springer, 440–457.
- [169] Ryo Asakura. 2023. Rust-Autograd. Retrieved from <https://github.com/raskr/rust-autograd>
- [170] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. 2019. PyTorch: An imperative style, high-performance deep learning library. *Advan. Neural Inf. Process. Syst.* 32 (2019), 8026–8037.
- [171] Le Trieu Phong, Yoshinori Aono, Takuya Hayashi, Lihua Wang, and Shihō Moriai. 2017. Privacy-preserving deep learning via additively homomorphic encryption. *IEEE Trans. Inf. Forens. Secur.* 13, 5 (2017), 1333–1345.
- [172] Se Eun Oh, Taiji Yang, Nate Mathews, James K. Holland, Mohammad Saidur Rahman, Nicholas Hopper, and Matthew Wright. 2022. DeepCoFFEA: Improved flow correlation attacks on Tor via metric learning and amplification. In *IEEE Symposium on Security and Privacy (SP'22)*. IEEE, 1915–1932.
- [173] Zhi Yang, Zhanhui Yuan, Shuyuan Jin, Xingyuan Chen, Lei Sun, Xuehui Du, Wenfa Li, and Hongqi Zhang. 2022. FSAFlow: Lightweight and fast dynamic path tracking and control for privacy protection on Android using hybrid analysis with state-reduction strategy. In *43rd IEEE Symposium on Security and Privacy (SP'22)*. 23–25.
- [174] Jinyuan Jia, Ahmed Salem, Michael Backes, Yang Zhang, and Neil Zhenqiang Gong. 2019. MemGuard: Defending against black-box membership inference attacks via adversarial examples. In *ACM SIGSAC Conference on Computer and Communications Security*. 259–274.
- [175] M. A. Poltavtseva and E. A. Rudnitskaya. 2023. Confidentiality of machine learning models. *Autom. Contr. Comput. Sci.* 57, 8 (2023), 975–982.
- [176] Fan Mo, Anastasia Borovykh, Mohammad Malekzadeh, Hamed Haddadi, and Soteris Demetriou. 2020. Layer-wise characterization of latent information leakage in federated learning. *arXiv preprint arXiv:2010.08762* (2020).
- [177] Fan Mo, Anastasia Borovykh, Mohammad Malekzadeh, Hamed Haddadi, and Soteris Demetriou. 2021. Quantifying information leakage from gradients. *arXiv preprint arXiv:2105.13929* (2021).
- [178] Fan Mo, Ali Shahin Shamsabadi, Kleomenis Katevas, Andrea Cavallaro, and Hamed Haddadi. 2019. Towards characterizing and limiting information exposure in DNN layers. *arXiv preprint arXiv:1907.06034* (2019).
- [179] Zhongshu Gu, Heqing Huang, Jialong Zhang, Dong Su, Ankita Lamba, Dimitrios Pendarakis, and Ian Molloy. 2018. Securing input data of deep learning inference systems via partitioned enclave execution. *arXiv preprint arXiv:1807.00969* (2018).
- [180] Stavros Volos, Kapil Vaswani, and Rodrigo Bruno. 2018. Graviton: Trusted execution environments on GPUs. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI'18)*. 681–696.
- [181] Zhongshu Gu, Heqing Huang, Jialong Zhang, Dong Su, Hani Jamjoom, Ankita Lamba, Dimitrios Pendarakis, and Ian Molloy. 2018. Confidential inference via ternary model partitioning. *arXiv preprint arXiv:1807.00969* (2018).
- [182] Krishna Giri Narra, Zhifeng Lin, Yongqin Wang, Keshav Balasubramaniam, and Murali Annavaram. 2019. Privacy-preserving inference in machine learning services using trusted execution environments. *arXiv preprint arXiv:1912.03485* (2019).
- [183] Xiaoli Zhang, Fengting Li, Zeyu Zhang, Qi Li, Cong Wang, and Jianping Wu. 2020. Enabling execution assurance of federated learning at untrusted participants. In *IEEE Conference on Computer Communications (INFOCOM'20)*. IEEE, 1877–1886.

- [184] Tyler Hunt, Zhipeng Jia, Vance Miller, Ariel Szekely, Yige Hu, Christopher J. Rossbach, and Emmett Witchel. 2020. Telekine: Secure computing with cloud GPUs. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI'20)*. 817–833.
- [185] Tarek Elgamal and Klara Nahrstedt. 2020. Serdab: An IoT framework for partitioning neural networks computation across multiple enclaves. In *20th IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing (CCGRID'20)*. IEEE, 519–528.
- [186] Akshay Gangal, Mengmei Ye, and Sheng Wei. 2020. HybridTEE: Secure mobile DNN execution using hybrid trusted execution environment. In *Asian Hardware Oriented Security and Trust Symposium (AsianHOST'20)*. IEEE, 1–6.
- [187] Yecheng Xiang, Yidi Wang, Hyunjong Choi, Mohsen Karimi, and Hyoseung Kim. 2021. AegisDNN: Dependable and timely execution of DNN tasks with SGX. In *IEEE Real-Time Systems Symposium (RTSS'21)*. IEEE, 68–81.
- [188] Lucjan Hanzlik, Yang Zhang, Kathrin Grosse, Ahmed Salem, Maximilian Augustin, Michael Backes, and Mario Fritz. 2021. MLCapsule: Guarded offline deployment of machine learning as a service. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 3300–3309.
- [189] Zhuang Liu, Ye Lu, Xueshuo Xie, Yaozheng Fang, Zhaolong Jian, and Tao Li. 2021. Trusted-DNN: A TrustZone-based adaptive isolation strategy for deep neural networks. In *ACM Turing Award Celebration Conference-China (ACM TURC'21)*. 67–71.
- [190] Jean-Baptiste Truong, William Gallagher, Tian Guo, and Robert J. Walls. 2021. Memory-efficient deep learning inference in trusted execution environments. *arXiv preprint arXiv:2104.15109* (2021).
- [191] Do Le Quoc and Christof Fetzer. 2021. SecFL: Confidential federated learning using TEEs. *arXiv preprint arXiv:2110.00981* (2021).
- [192] Lucien K. L. Ng, Sherman S. M. Chow, Anna P. Y. Woo, Donald P. H. Wong, and Yongjun Zhao. 2021. GOTEN: GPU-outsourcing trusted execution of neural network training. In *AAAI Conference on Artificial Intelligence*. 14876–14883.
- [193] Chengliang Zhang, Junzhe Xia, Baichen Yang, Huancheng Puyang, Wei Wang, Ruichuan Chen, Istemi Ekin Akkus, Paarijaat Aditya, and Feng Yan. 2021. Citadel: Protecting data privacy and model confidentiality for collaborative learning. In *ACM Symposium on Cloud Computing*. 546–561.
- [194] Renju Liu, Luis Garcia, Zaoxing Liu, Botong Ou, and Mani Srivastava. 2021. SecDeep: Secure and performant on-device deep learning inference framework for mobile and IoT devices. In *International Conference on Internet-of-Things Design and Implementation*. 67–79.
- [195] Saerom Park, Seongmin Kim, and Yeon-sup Lim. 2022. Fairness audit of machine learning models with confidential computing. In *ACM Web Conference*. 3488–3499.
- [196] Weizhe Hua, Muhammad Umar, Zhiru Zhang, and G. Edward Suh. 2022. GuardNN: Secure accelerator architecture for privacy-preserving deep learning. In *59th ACM/IEEE Design Automation Conference*. 349–354.
- [197] Tianxiang Shen, Ji Qi, Jianyu Jiang, Xian Wang, Siyuan Wen, Xusheng Chen, Shixiong Zhao, Sen Wang, Li Chen, Xipao Luo, Fengwei Zhang, and Heming Cui. 2022. SOTER: Guarding Black-box inference for general neural networks at the edge. In *USENIX Annual Technical Conference (USENIX ATC'22)*. USENIX Association, Carlsbad, CA, 723–738. <https://www.usenix.org/conference/atc22/presentation/shen>
- [198] Yunjie Deng, Chenxu Wang, Shunchang Yu, Shiqing Liu, Zhenyu Ning, Kevin Leach, Jin Li, Shoumeng Yan, Zhengyu He, Jiannong Cao, and Fengwei Zhang. 2022. StrongBox: A GPU TEE on arm endpoints. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS'22)*. Association for Computing Machinery, New York, NY, USA, 769–783. <https://doi.org/10.1145/3548606.3560627>
- [199] Jonas Sander, Sebastian Berndt, Ida Bruhns, and Thomas Eisenbarth. 2023. DASH: Accelerating distributed private machine learning inference with arithmetic garbled circuits. *arXiv preprint arXiv:2302.06361* (2023).
- [200] Kapil Vaswani, Stavros Volos, Cedric Fournet, Antonio Nino Diaz, Ken Gordon, Balaji Vembu, Sam Webster, David Chisnall, Saurabh Kulkarni, Graham Cunningham, Richard Osborne, and Daniel Wilkinson. 2023. Confidential computing within an AI accelerator. In *USENIX Annual Technical Conference (USENIX ATC'23)*. USENIX Association, Boston, MA, 501–518. <https://www.usenix.org/conference/atc23/presentation/vaswani>
- [201] Haohui Mai, Jiacheng Zhao, Hongren Zheng, Yiyang Zhao, Zibin Liu, Mingyu Gao, Cong Wang, Huimin Cui, Xiaobing Feng, and Christos Kozyrakis. 2023. Honeycomb: Secure and efficient GPU executions via static validation. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI'23)*. 155–172.
- [202] Md Shihabul Islam, Mahmoud Zamani, Chung Hwan Kim, Latifur Khan, and Kevin W. Hamlen. 2023. Confidential execution of deep learning inference at the untrusted edge with ARM TrustZone. In *13th ACM Conference on Data and Application Security and Privacy*. 153–164.
- [203] Jinwen Wang, Yujie Wang, and Ning Zhang. 2023. Secure and timely GPU execution in cyber-physical systems. In *ACM SIGSAC Conference on Computer and Communications Security*. 2591–2605.
- [204] Kyungmi Lee, Mengjia Yan, Joel Emer, and Anantha Chandrakasan. 2023. SecureLoop: Design space exploration of secure DNN accelerators. In *56th Annual IEEE/ACM International Symposium on Microarchitecture*. 194–208.

- [205] Zahra Tarkhani and Anil Madhavapeddy. 2023. Information flow tracking for heterogeneous compartmentalized software. In *26th International Symposium on Research in Attacks, Intrusions and Defenses*. 564–579.
- [206] Jiasi Weng, Shenglong Yao, Yuefeng Du, Junjie Huang, Jian Weng, and Cong Wang. 2024. Proof of unlearning: Definitions and instantiation. *IEEE Trans. Inf. Forens. Secur.* (2024).
- [207] Chenxu Wang, Fengwei Zhang, Yunjie Deng, Kevin Leach, Jiannong Cao, Zhenyu Ning, Shoumeng Yan, and Zhengyu He. 2024. CAGE: Complementing Arm CCA with GPU Extensions. In *Network and Distributed System Security Symposium (NDSS'24)*.
- [208] Mohammad Fakhruddin Babar and Monowar Hasan. 2022. Real-time scheduling of TrustZone-enabled DNN workloads. In *4th Workshop on CPS & IoT Security and Privacy*. 63–69.
- [209] Fan Mo, Hamed Haddadi, Kleomenis Katevas, Eduard Marin, Diego Perino, and Nicolas Kourtellis. 2022. PPFL: Enhancing privacy in federated learning with confidential computing. *GetMobile: Mob. Comput. Commun.* 25, 4 (2022), 35–38.
- [210] Peter M. VanNostrand, Ioannis Kyriazis, Michelle Cheng, Tian Guo, and Robert J. Walls. 2019. Confidential deep learning: Executing proprietary models on untrusted devices. *arXiv preprint arXiv:1908.10730* (2019).
- [211] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. 2012. ImageNet classification with deep convolutional neural networks. *Advan. Neural Inf. Process. Syst.* 25 (2012), 1097–1105.
- [212] Mark Zhao, Mingyu Gao, and Christos Kozyrakis. 2022. ShEF: Shielded enclaves for cloud FPGAs. In *27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. 1070–1085.
- [213] Ke Xia, Yukui Luo, Xiaolin Xu, and Sheng Wei. 2021. SGX-FPGA: Trusted execution environment for CPU-FPGA heterogeneous architecture. In *58th ACM/IEEE Design Automation Conference (DAC'21)*. IEEE, 301–306.
- [214] Heejin Park and Felix Xiao Zhu Lin. 2021. Safe and practical GPU acceleration in TrustZone. *arXiv preprint arXiv:2111.03065* (2021).
- [215] NVIDIA. NVIDIA H100 Tensor Core GPU Architecture. Retrieved 24-March-2024 from <https://nvdam.widen.net/s/9bz6dw7dqr/gtc22-whitepaperhopper>
- [216] Michael Andersch, Greg Palmer, Ronny Krashinsky, Nick Stam, Vishal Mehta, Gonzalo Brito, and Sridhar Ramaswamy. 2022. Nvidia hopper architecture in-depth. Retrieved 08-June-2024 from <https://developer.nvidia.com/blog/nvidia-hopper-architecture-in-depth/>
- [217] Zahra Tarkhani. 2022. *Secure Programming with Dispersed Compartments*. Ph.D. Dissertation. University of Cambridge.
- [218] Xuhong Li, Haoyi Xiong, Xingjian Li, Xuanyu Wu, Xiao Zhang, Ji Liu, Jiang Bian, and Dejing Dou. 2021. Interpretable deep learning: Interpretation, interpretability, trustworthiness, and beyond. *arXiv preprint arXiv:2103.10689* (2021).
- [219] Florian Tramèr, Alexey Kurakin, Nicolas Papernot, Ian Goodfellow, Dan Boneh, and Patrick McDaniel. 2018. Ensemble adversarial training: Attacks and defenses. In *International Conference on Learning Representations*.
- [220] Amrita Roy Chowdhury, Chuan Guo, Somesh Jha, and Laurens van der Maaten. 2021. EIFFeL: Ensuring integrity for federated learning. *arXiv preprint arXiv:2112.12727* (2021).
- [221] Rajeev Motwani and Prabhakar Raghavan. 1996. Randomized algorithms. *ACM Comput. Surv.* 28, 1 (1996), 33–37.
- [222] Rui Zhang, Jian Liu, Yuan Ding, Zhibo Wang, Qingbiao Wu, and Kui Ren. 2022. “Adversarial examples” for proof-of-learning. In *IEEE Symposium on Security and Privacy (SP'22)*. IEEE, 1408–1422.
- [223] Ralph C. Merkle. 1980. Protocols for public key cryptosystems. In *IEEE Symposium on Security and Privacy*. IEEE, 122–122.
- [224] Nils Lukas, Edward Jiang, Xinda Li, and Florian Kerschbaum. 2022. SoK: How robust is image classification deep neural network watermarking? In *IEEE Symposium on Security and Privacy (SP'22)*. IEEE, 787–804.
- [225] Ralph C. Merkle. 1989. A certified digital signature. In *Conference on the Theory and Application of Cryptology*. Springer, 218–238.
- [226] Fred Cohen. 1987. A cryptographic checksum for integrity protection. *Comput. Secur.* 6, 6 (1987), 505–510.
- [227] Nikolaos Karapanos, Alexandros Filios, Raluca Ada Popa, and Srdjan Capkun. 2016. Verena: End-to-end integrity protection for web applications. In *IEEE Symposium on Security and Privacy (SP'16)*. IEEE, 895–913.
- [228] Carla E. Brodley and Mark A. Friedl. 1999. Identifying mislabeled training data. *J. Artif. Intell. Res.* 11 (1999), 131–167.
- [229] Xu Chu, Ihab F. Ilyas, Sanjay Krishnan, and Jiannan Wang. 2016. Data cleaning: Overview and emerging challenges. In *International Conference on Management of Data*. 2201–2206.
- [230] Dara Bahri, Heinrich Jiang, and Maya Gupta. 2020. Deep k-NN for noisy labels. In *International Conference on Machine Learning*. PMLR, 540–550.
- [231] Zahra Tarkhani and Anil Madhavapeddy. 2020. Enclave-aware compartmentalization and secure sharing with Sirius. *arXiv preprint arXiv:2009.01869* (2020).
- [232] Sanchuan Chen, Zhiqiang Lin, and Yinqian Zhang. 2023. Controlled data races in enclaves: Attacks and detection. In *32nd USENIX Security Symposium (USENIX Security'23)*. USENIX Association.

- [233] Intel. 2023. Intel® TDX Full Report. Retrieved 08-June-2024 from https://services.google.com/fh/files/misc/intel_tdx_-_full_report_041423.pdf
- [234] Intel. 2024. INTEL-SA-01036: Intel® TDX Module Software Advisory. Retrieved from <https://www.intel.com/content/www/us/en/security-center/advisory/intel-sa-01036.html>
- [235] Zahra Tarkhani, Anil Madhavapeddy, and Richard Mortier. 2019. Snape: The dark art of handling heterogeneous enclaves. In *2nd International Workshop on Edge Systems, Analytics and Networking*. 48–53.
- [236] Dariusz Suciu, Stephen McLaughlin, Laurent Simon, and Radu Sion. 2020. Horizontal privilege escalation in trusted applications. In *29th USENIX Security Symposium (USENIX Security'20)*.
- [237] Stephen Checkoway and Hovav Shacham. 2013. *Iago Attacks: Why the System Call API Is a Bad Untrusted RPC Interface*. Vol. 41. ACM.
- [238] Jo Van Bulck, David Oswald, Eduard Marin, Abdulla Aldoseri, Flavio D. Garcia, and Frank Piessens. 2019. A tale of two worlds: Assessing the vulnerability of enclave shielding runtimes. In *ACM SIGSAC Conference on Computer and Communications Security*. 1741–1758.
- [239] Samuel Weiser, Luca Mayr, Michael Schwarz, and Daniel Gruss. 2019. SGXJail: Defeating enclave malware via confinement. In *22nd International Symposium on Research in Attacks, Intrusions and Defenses (RAID'19)*. 353–366.
- [240] Yuntao Liu and Ankur Srivastava. 2020. GANRED: GAN-based reverse engineering of DNNs via cache side-channel. In *ACM SIGSAC Conference on Cloud Computing Security Workshop*. 41–52.
- [241] Yun Xiang, Zhuangzhi Chen, Zuohui Chen, Zebin Fang, Haiyang Hao, Jinyin Chen, Yi Liu, Zhefu Wu, Qi Xuan, and Xiaoni Yang. 2020. Open DNN box by power side-channel attack. *IEEE Trans. Circ. Syst. II: Express Briefs* 67, 11 (2020), 2717–2721.
- [242] Lejla Batina, Shivam Bhasin, Dirmanto Jap, and Stjepan Picek. 2019. CSINN: Reverse engineering of neural network architectures through electromagnetic side channel. In *28th USENIX Security Symposium (USENIX Security'19)*. 515–532.
- [243] Wenting Zheng, Ankur Dave, Jethro G. Beekman, Raluca Ada Popa, Joseph E. Gonzalez, and Ion Stoica. 2017. Opaque: An oblivious and encrypted distributed analytics platform. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI'17)*. 283–298.
- [244] Wenjia Zhao, Kangjie Lu, Yong Qi, and Saiyu Qi. 2020. MPTEE: Bringing flexible and efficient memory protection to Intel SGX. In *15th European Conference on Computer Systems*. 1–15.
- [245] Youren Shen, Hongliang Tian, Yu Chen, Kang Chen, Runji Wang, Yi Xu, Yubin Xia, and Shoumeng Yan. 2020. Occlum: Secure and efficient multitasking inside a single enclave of Intel SGX. In *25th International Conference on Architectural Support for Programming Languages and Operating Systems*. 955–970.
- [246] Insu Jang, Adrian Tang, Taehoon Kim, Simha Sethumadhavan, and Jaehyuk Huh. 2019. Heterogeneous isolated execution for commodity GPUs. In *24th International Conference on Architectural Support for Programming Languages and Operating Systems*. 455–468.
- [247] Ayaz Akram, Venkatesh Akella, Sean Peisert, and Jason Lowe-Power. 2022. SoK: Limitations of confidential computing via TEEs for high-performance compute systems. In *IEEE International Symposium on Secure and Private Execution Environment Design (SEED'22)*. IEEE, 121–132.
- [248] Sandra Siby, Sina Abdollahi, Mohammad Maheri, Marios Kogias, and Hamed Haddadi. 2024. Guarantees: Towards attestable and private ML with CCA. In *4th Workshop on Machine Learning and Systems*. 1–9.
- [249] Shixuan Zhao, Mengyuan Li, Yinqian Zhang, and Zhiqiang Lin. 2022. vSGX: Virtualizing SGX enclaves on AMD SEV. In *IEEE Symposium on Security and Privacy (SP'22)*. IEEE, 321–336.
- [250] NXP. ASUG-i.MX Android Security User's Guide. Retrieved from https://www.nxp.com/docs/en/user-guide/IMX_ANDROID_SECURITY_USERS_GUIDE.pdf
- [251] Thomas Steinke, Milad Nasr, and Matthew Jagielski. 2023. Privacy auditing with one (1) training run. *arXiv preprint arXiv:2305.08846* (2023).
- [252] Martin Abadi, Andy Chu, Ian Goodfellow, H. Brendan McMahan, Ilya Mironov, Kunal Talwar, and Li Zhang. 2016. Deep learning with differential privacy. In *ACM SIGSAC Conference on Computer and Communications Security*. 308–318.
- [253] Yilun Xu, Shengjia Zhao, Jiaming Song, Russell Stewart, and Stefano Ermon. 2019. A theory of usable information under computational constraints. In *International Conference on Learning Representations*.
- [254] NVIDIA. 2024. NVIDIA Blackwell platform arrives to power a new era of computing. Retrieved from <https://nvidianews.nvidia.com/news/nvidia-blackwell-platform-arrives-to-power-a-new-era-of-computing>
- [255] Pau-Chen Cheng, Wojciech Ozga, Enriquillo Valdez, Salman Ahmed, Zhongshu Gu, Hani Jamjoom, Hubertus Franke, and James Bottomley. 2024. Intel TDX demystified: A top-down approach. *Comput. Surv.* 56, 9 (2024), 1–33.
- [256] Abu Sebastian, Manuel Le Gallo, Riduan Khaddam-Aljameh, and Evangelos Eleftheriou. 2020. Memory devices and applications for in-memory computing. *Nat. Nanotechnol.* 15, 7 (2020), 529–544.

- [257] Daniele Ielmini and H.-S. Philip Wong. 2018. In-memory computing with resistive switching devices. *Nat. Electron.* 1, 6 (2018), 333–343.
- [258] Evangelos Eleftheriou, Manuel Le Gallo, S. R. Nandakumar, Christophe Piveteau, Irem Boybat, Vinay Joshi, Riduan Khaddam-Aljameh, Martino Dazzi, Iason Giannopoulos, Geethan Karunaratne, et al. 2019. Deep learning acceleration based on in-memory computing. *IBM J. Res. Devel.* 63, 6 (2019), 7–1.
- [259] Stanisław Woźniak, Angeliki Pantazi, Thomas Bohnstingl, and Evangelos Eleftheriou. 2020. Deep learning incorporating biologically inspired neural dynamics and in-memory computing. *Nat. Mach. Intell.* 2, 6 (2020), 325–336.
- [260] Igor Fedorov, Ryan P. Adams, Matthew Mattina, and Paul Whatmough. 2019. Sparse: Sparse architecture search for CNNs on resource-constrained microcontrollers. *Advan. Neural Inf. Process. Syst.* 32 (2019).
- [261] Colby Banbury, Chuteng Zhou, Igor Fedorov, Ramon Matas, Urmish Thakker, Dibakar Gope, Vijay Janapa Reddi, Matthew Mattina, and Paul Whatmough. 2021. MicroNets: Neural network architectures for deploying TinyML applications on commodity microcontrollers. *Proc. Mach. Learn. Syst.* 3 (2021), 517–532.
- [262] Bharath Sudharsan, John G. Breslin, and Muhammad Intizar Ali. 2021. ML-MCU: A framework to train ML classifiers on MCU-based IoT edge devices. *IEEE Internet Things J.* (2021).
- [263] Rong Dai, Li Shen, Fengxiang He, Xinmei Tian, and Dacheng Tao. 2022. DisPFL: Towards communication-efficient personalized federated learning via decentralized sparse training. *arXiv preprint arXiv:2206.00187* (2022).
- [264] Ang Li, Jingwei Sun, Pengcheng Li, Yu Pu, Hai Li, and Yiran Chen. 2021. Hermes: An efficient federated learning framework for heterogeneous mobile clients. In *27th Annual International Conference on Mobile Computing and Networking*. 420–437.
- [265] Enmao Diao, Jie Ding, and Vahid Tarokh. 2020. HeteroFL: Computation and communication efficient federated learning for heterogeneous clients. In *International Conference on Learning Representations*.
- [266] Jie Zhang, Song Guo, Xiaosong Ma, Haozhao Wang, Wenchao Xu, and Feijie Wu. 2021. Parameterized knowledge transfer for personalized federated learning. *Advan. Neural Inf. Process. Syst.* 34 (2021), 10092–10104.
- [267] Nikolaj Volgushev, Malte Schwarzkopf, Ben Getchell, Mayank Varia, Andrei Lapets, and Azer Bestavros. 2019. Conclave: Secure multi-party computation on big data. In *14th European Conference on Computer Systems (EuroSys'19)*. 1–18.
- [268] Jatinder Singh and Jennifer Cobbe. 2021. Do Le Quoc, and Zahra Tarkhani. 2021. Enclaves in the clouds: Legal considerations and broader implications. *Commun. ACM* 64, 5 (2021), 42–51.
- [269] Weizhe Hua, Zhiru Zhang, and G. Edward Suh. 2018. Reverse engineering convolutional neural networks through side-channel information leaks. In *55th Annual Design Automation Conference*. 1–6.
- [270] Mengjia Yan, Christopher Fletcher, and Josep Torrellas. 2020. Cache telepathy: Leveraging shared resource attacks to learn DNN architectures. In *USENIX Security Symposium*.
- [271] Honggang Yu, Haocheng Ma, Kaichen Yang, Yiqiang Zhao, and Yier Jin. 2020. DeepEM: Deep neural networks model recovery through EM side-channel information leakage. In *IEEE International Symposium on Hardware Oriented Security and Trust (HOST'20)*. IEEE, 209–218.
- [272] Junyi Wei, Yicheng Zhang, Zhe Zhou, Zhou Li, and Mohammad Abdullah Al Faruque. 2020. Leaky DNN: Stealing deep-learning model secret with GPU context-switching side-channel. In *50th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN'20)*. IEEE, 125–137.
- [273] Hyunyoung Oh, Adil Ahmad, Seonghyun Park, Byoungyoung Lee, and Yunheung Paek. 2020. TRUSTORE: Side-channel resistant storage for SGX using Intel hybrid CPU-FPGA. In *ACM SIGSAC Conference on Computer and Communications Security*. 1903–1918.
- [274] Bo Liu, Ming Ding, Sina Shaham, Wenny Rahayu, Farhad Farokhi, and Zihuai Lin. 2021. When machine learning meets privacy: A survey and outlook. *ACM Comput. Surv.* 54, 2 (2021), 1–36.
- [275] Hongsheng Hu, Zoran Salcic, Lichao Sun, Gillian Dobbie, Philip S. Yu, and Xuyun Zhang. 2022. Membership inference attacks on machine learning: A survey. *ACM Comput. Surv.* 54, 11s (2022), 1–37.
- [276] Ahmed Salem, Giovanni Cherubin, David Evans, Boris Köpf, Andrew Paverd, Anshuman Suri, Shruti Tople, and Santiago Zanella-Béguelin. 2023. SoK: Let the privacy games begin! A unified treatment of data inference privacy in machine learning. In *IEEE Symposium on Security and Privacy (SP'23)*. IEEE, 327–345.
- [277] Maria Rigaki and Sebastian Garcia. 2023. A survey of privacy attacks in machine learning. *Comput. Surv.* 56, 4 (2023), 1–34.
- [278] Mohammad Fakhruddin Babar and Monowar Hasan. 2023. Trusted deep neural execution—A survey. *IEEE Access* 11 (2023), 45736–45748.
- [279] Intersoft Consulting. 2024. General Data Protection Regulation (GDPR). Retrieved 08-June-2024 from <https://gdpr-info.eu/>
- [280] State of California Department of Justice. 2024. California Consumer Privacy Act (CCPA). Retrieved 08-june-2024 from <https://oag.ca.gov/privacy/ccpa>

- [281] Future of Life Institute. 2024. The EU Artificial Intelligence Act. Retrieved 08-June-2024 from <https://artificialintelligenceact.eu/>
- [282] Microsoft. 2024. Virtualization-based Security (VBS) Enclaves. Retrieved 08-June-2024 from <https://learn.microsoft.com/en-us/windows/win32/trustedexecution/vbs-enclaves>

Received 2 April 2023; revised 20 May 2024; accepted 26 May 2024